9

Newton iteration

We mentioned on pages 218–219 Newton's method for approximating roots of polynomials. It has become a staple of numerical computation, and seen many generalizations and improvements over the years. But what does this decidedly *continuous*, *approximative* method, computing values that are closer and closer to some real root, have to do with the *discrete*, *exact* calculations prevalent in computer algebra? There is a somewhat counter-intuitive notion of *closeness* for integers (and polynomials), corresponding to divisibility by higher and higher powers of a fixed prime. Newton iteration works just beautifully in this purely algebraic setting.

We start by using it to find a custom-Taylored division algorithm that is about as fast as multiplication, and then describe its use for finding roots of polynomials. Finally, we describe a common framework—valuations—into which both the analytical method over the real numbers and our symbolic version fit. In Chapter 15, we will apply Newton's method to the factorization of polynomials; it is then called Hensel lifting.

9.1. Division with remainder using Newton iteration

Integers and polynomials over a field form a Euclidean domain when the absolute value and the polynomial degree, respectively, is used as the Euclidean function. This means that for all a, b with $b \neq 0$ there exist unique q, r such that $a = qb + r$ where $0 \leq r < |b|$ in the integer case and $\deg r < \deg b$ in the polynomial case. The division problem is then to find q, r, given a, b.

The "classical" (or "synthetic") division algorithm 2.5 requires $O(n^2)$ (word or field) operations for inputs of size n. We will now see how this can be improved to $O(\mathsf{M}(n))$, where M is a multiplication time. We only discuss the polynomial case since it is somewhat easier, and then briefly indicate the ideas in the integer case. Let D be a ring (commutative, with 1) and $a, b \in D[x]$ two polynomials of degree n and m, respectively. We assume that $m \leq n$ and that b is monic. We wish to find polynomials q and r in $D[x]$ satisfying $a = qb + r$ with $\deg r < \deg b$

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

(where, as usual, we assume that the zero polynomial has degree $-\infty$). Since b is monic, such q, r exist uniquely even if D is not a field (Section 2.4).

Substituting $1/x$ for the variable x and multiplying by x^n, we obtain

$$x^n a\Big(\frac{1}{x}\Big) = \Big(x^{n-m} q\Big(\frac{1}{x}\Big)\Big) \cdot \Big(x^m b\Big(\frac{1}{x}\Big)\Big) + x^{n-m+1}\Big(x^{m-1} r\Big(\frac{1}{x}\Big)\Big). \tag{1}$$

We define the **reversal** of a as $\mathrm{rev}_k(a) = x^k a(1/x)$. When $k = n$, this is the polynomial with the coefficients of a reversed, that is, if $a = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, then

$$\mathrm{rev}(a) = \mathrm{rev}_n(a) = a_0 x^n + a_1 x^{n-1} + \cdots + a_{n-1}x + a_n$$

Equation (1) now reads

$$\mathrm{rev}_n(a) = \mathrm{rev}_{n-m}(q) \cdot \mathrm{rev}_m(b) + x^{n-m+1}\,\mathrm{rev}_{m-1}(r),$$

and therefore,

$$\mathrm{rev}_n(a) \equiv \mathrm{rev}_{n-m}(q) \cdot \mathrm{rev}_m(b) \bmod x^{n-m+1}.$$

We note that $\mathrm{rev}_m(b)$ has constant coefficient 1 and thus is invertible modulo x^{n-m+1}, by Theorem 4.1. Hence we find

$$\mathrm{rev}_{n-m}(q) \equiv \mathrm{rev}_n(a) \cdot \mathrm{rev}_m(b)^{-1} \bmod x^{n-m+1}, \tag{2}$$

and obtain $q = \mathrm{rev}_{n-m}(\mathrm{rev}_{n-m}(q))$ and $r = a - q \cdot b$.

EXAMPLE 9.1. Let $a = 5x^5 + 4x^4 + 3x^3 + 2x^2 + x$ and $b = x^2 + 2x + 3$ be polynomials in $\mathbb{F}_7[x]$. Then

$$\mathrm{rev}_5(a) = x^4 + 2x^3 + 3x^2 + 4x + 5, \quad \mathrm{rev}_2(b) = 3x^2 + 2x + 1.$$

We claim that $\mathrm{rev}_2(b)^{-1} \equiv 4x^3 + x^2 + 5x + 1 \bmod x^4$ in $\mathbb{F}_7[x]$, and indeed

$$(3x^2 + 2x + 1)(4x^3 + x^2 + 5x + 1) = 5x^5 + 4x^4 + 1 \equiv 1 \bmod x^4.$$

It follows that

$$\mathrm{rev}_3(q) \equiv (x^4 + 2x^3 + 3x^2 + 4x + 5)(4x^3 + x^2 + 5x + 1) \equiv 6x^3 + x + 5 \bmod x^4.$$

Thus $q = 5x^3 + x^2 + 6$ and

$$r = a - qb = 5x^5 + 4x^4 + 3x^3 + 2x^2 + x - (5x^3 + x^2 + 6)(x^2 + 2x + 3) = 3x + 3. \diamond$$

So now we have to solve the problem of finding, from a given $f \in D[x]$ and $l \in \mathbb{N}$ with $f(0) = 1$, a $g \in D[x]$ satisfying $fg \equiv 1 \bmod x^l$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

From numerical analysis, we recall that Newton iteration involves computing successive approximations to solutions of $\varphi(g) = 0$. From a suitable initial approximation g_0, subsequent approximations are computed using

$$g_{i+1} = g_i - \frac{\varphi(g_i)}{\varphi'(g_i)}, \tag{3}$$

where φ' is the derivative of φ. This corresponds to intersecting the tangent with an axis, as illustrated in Figure 9.1, or in other words, to replacing φ by its "linearization" or two-term Taylor expansion at that point. For the task at hand, we want to find (or approximate) a root of $1/g - f = 0$. The Newton iteration step is

$$g_{i+1} = g_i - \frac{1/g_i - f}{-1/g_i^2} = 2g_i - fg_i^2.$$

The following theorem tells us a good initial approximation and shows us that this method converges "quickly" to a solution also in our algebraic setting.

THEOREM 9.2.
Let D be a ring (commutative, with 1), $f, g_0, g_1, \ldots \in D[x]$, with $f(0) = 1$, $g_0 = 1$, and $g_{i+1} \equiv 2g_i - fg_i^2 \bmod x^{2^{i+1}}$, for all i. Then $fg_i \equiv 1 \bmod x^{2^i}$ for all $i \geq 0$.

PROOF. The proof is by induction on i. For $i = 0$ we have

$$fg_0 \equiv f(0)g_0 \equiv 1 \cdot 1 \equiv 1 \bmod x^{2^0}.$$

For the induction step, we find

$$1 - fg_{i+1} \equiv 1 - f(2g_i - fg_i^2) \equiv 1 - 2fg_i + f^2g_i^2 \equiv (1 - fg_i)^2 \equiv 0 \bmod x^{2^{i+1}}. \quad \square$$

We obtain the following algorithm to compute the inverse of $f \bmod x^l$. We denote by log the binary logarithm.

ALGORITHM 9.3 Inversion using Newton iteration.
Input: $f \in D[x]$ with $f(0) = 1$, and $l \in \mathbb{N}$.
Output: $g \in D[x]$ satisfying $fg \equiv 1 \bmod x^l$.

1. $g_0 \longleftarrow 1, \quad r \longleftarrow \lceil \log l \rceil$
2. **for** $i = 1, \ldots, r$ **do** $g_i \longleftarrow (2g_{i-1} - fg_{i-1}^2) \text{ rem } x^{2^i}$
3. **return** g_r

If we want to compute the inverse of a polynomial $f \in D[x]$ with $f(0)$ a unit different from 1, then we set $g_0 = f(0)^{-1}$ in step 1. If $f(0)$ is not a unit, then no inverse of f modulo x^l exists, since $fg \equiv 1 \bmod x^l$ implies that $f(0) \cdot g(0) = 1$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

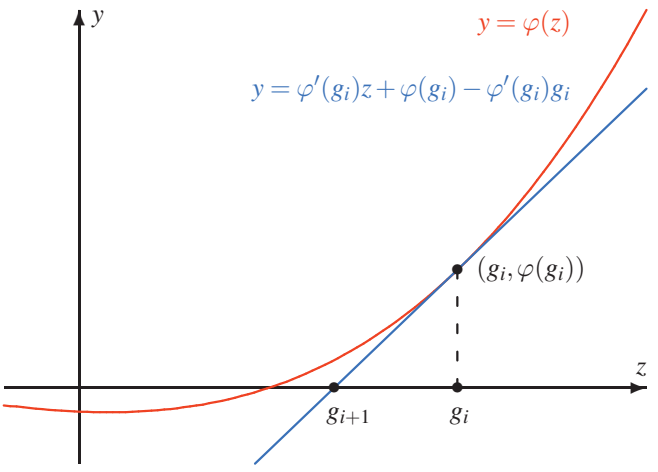


FIGURE 9.1: Newton iteration over the reals.

EXAMPLE 9.1 (continued). Let $f = 3x^2 + 2x + 1 \in \mathbb{F}_7[x]$ and $l = 4$. Then Algorithm 9.3 computes $g_0 = 1$, $r = 2$, and

$$\begin{aligned} g_1 &\equiv 2g_0 - fg_0^2 = 2 - (3x^2 + 2x + 1) \equiv 5x + 1 \bmod x^2, \\ g &\equiv 2g_1 - fg_1^2 = 2(5x+1) - (3x^2 + 2x + 1)(5x+1)^2 \\ &= 2x^4 + 4x^3 + x^2 + 5x + 1 \equiv 4x^3 + x^2 + 5x + 1 \bmod x^4. \end{aligned}$$

We have already checked in Example 9.1 that indeed $fg \equiv 1 \bmod x^4$. ◇

THEOREM 9.4.
Algorithm 9.3 correctly computes the inverse of f modulo x^l. If $l = 2^r$ is a power of 2, then it uses at most $3\mathsf{M}(l) + l \in O(\mathsf{M}(l))$ arithmetic operations in D.

PROOF. Correctness follows from Theorem 9.2 and the fact that x^l divides x^{2^r}. In step 2, all powers of x up to 2^i can be dropped, and since $g_i \equiv g_{i-1} \cdot (2 - fg_{i-1}) \equiv g_{i-1} \bmod x^{2^{i-1}}$, also the powers of x less than 2^{i-1}. The cost for one iteration of step 2 is $\mathsf{M}(2^{i-1})$ for the computation of g_{i-1}^2, $\mathsf{M}(2^i)$ for the product $fg_{i-1}^2 \bmod x^{2^i}$, and then the negative of the upper half of fg_{i-1}^2 modulo x^{2^i} is the upper half of g_i, taking 2^{i-1} operations. Thus we have $\mathsf{M}(2^i) + \mathsf{M}(2^{i-1}) + 2^{i-1} \leq \frac{3}{2}\mathsf{M}(2^i) + 2^{i-1}$ in step 2, and the total running time is

$$\sum_{1 \leq i \leq r} \left(\frac{3}{2}\mathsf{M}(2^i) + 2^{i-1} \right) \leq \left(\frac{3}{2}\mathsf{M}(2^r) + 2^{r-1} \right) \sum_{1 \leq i \leq r} 2^{i-r} < 3\mathsf{M}(2^r) + 2^r = 3\mathsf{M}(l) + l,$$

where we have used $2\mathsf{M}(n) \leq \mathsf{M}(2n)$ for all $n \in \mathbb{N}$. □

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

If l is not a power of 2, then the above algorithm computes too many coefficients of the inverse. Exercise 9.6 gives a better algorithm with essentially the same running time bound in this general case.

ALGORITHM 9.5 Fast division with remainder.
Input: $a, b \in D[x]$, where D is a ring (commutative, with 1) and $b \neq 0$ is monic.
Output: $q, r \in D[x]$ such that $a = qb + r$ and $\deg r < \deg b$.

1. **if** $\deg a < \deg b$ **then return** $q = 0$ and $r = a$
2. $m \longleftarrow \deg a - \deg b$
   call Algorithm 9.3 to compute the inverse of $\mathrm{rev}_{\deg b}(b) \in D[x]$ modulo x^{m+1}
3. $q^* \longleftarrow \mathrm{rev}_{\deg a}(a) \cdot \mathrm{rev}_{\deg b}(b)^{-1} \bmod x^{m+1}$
4. **return** $q = \mathrm{rev}_m(q^*)$ and $r = a - bq$

THEOREM 9.6.
Let D be a ring (commutative, with 1). Division with remainder of a polynomial $a \in D[x]$ of degree $n+m$ by a monic polynomial $b \in D[x]$ of degree n, where $n \geq m \in \mathbb{N}$, can be done using $4\mathsf{M}(m) + \mathsf{M}(n) + O(n)$ ring operations.

PROOF. Let $a = qb + r$, with $q, r \in D[x]$ such that $\deg r < n$. Then we have $\deg q = \deg a - \deg b = m$. The correctness of Algorithm 9.5 follows from the discussion at the beginning of the section. Using Exercises 8.34 and 9.6, we have at most $3\mathsf{M}(m) + O(m)$ operations in step 2 of Algorithm 9.5, $\mathsf{M}(m) + O(m)$ in step 3, and finally $\mathsf{M}(n) + O(n)$ in step 4; only the lower part of $a - qb$ has to be computed since $\deg r < \deg b$. □

It may seem circular to use an algorithm that uses the rem operation to perform division. However, we are only using the rem operation to truncate the polynomial. It is similar to finding the quotient and remainder of a large number written in base 10 when divided by 10000. Division in this special case costs no operations.

What is the number of word operations for division with remainder in $\mathbb{Z}[x]$? If b is monic and a, b have max-norm $\|a\|_\infty, \|b\|_\infty < 2^l$, then Exercise 6.44 shows that $\|q\|_\infty, \|r\|_\infty < 2^{nl}$. Exercise 9.15 shows that all intermediate results in Algorithm 9.3 have coefficients of length $O(nl)$, and hence the cost for division using Newton inversion is $O(\mathsf{M}(n)\mathsf{M}(nl))$ or $O^\sim(n^2 l)$ word operations. Since the output size is $O(n^2 l)$ and Exercise 6.44 also shows that this bound can be achieved, the running time is—up to logarithmic factors—asymptotically optimal. A similar statement also holds for division with remainder in $F[y][x]$ for a field F.

Exercises 8.21 and 9.14 discuss slightly faster algorithms for **exact division**, where the remainder is known to be zero in advance.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

COROLLARY 9.7.
Let D be a ring (commutative, with 1) and $f \in D[x]$ monic of degree n. Then one multiplication in the residue class ring $D[x]/\langle f \rangle$ can be done using $6\mathsf{M}(n) + O(n)$ or $O(\mathsf{M}(n))$ arithmetic operations in D.

If several divisions by or multiplications modulo the same f have to be performed, then we may precompute $f^* = \mathrm{rev}_n(f)^{-1} \bmod x^n$ and store it, and the time for one division or one modular multiplication drops to at most $2\mathsf{M}(n) + O(n)$ and $3\mathsf{M}(n) + O(n)$, respectively. When using FFT multiplication, then also $\mathrm{DFT}_\omega(f)$ and $\mathrm{DFT}_\omega(f^*)$, where ω is an appropriate root of unity, may be precomputed and stored, thus further saving two FFTs per modular multiplication.

Now we want to divide with remainder an integer a by a nonzero integer b. Taking reversals with respect to the binary representation does not work here because of the carries. Instead, *numerical* Newton iteration over $\mathbb{R}$ is used to compute an approximation g to $1/b \in \mathbb{Q}$ with sufficient accuracy, then q is obtained by rounding ag to the nearest integer, and finally $r = a - qb$. The quotient may need to be adjusted by ± 1 if $r \geq b$ or $r < 0$. To avoid fractions, things are scaled appropriately by multiplying by powers of 2.

THEOREM 9.8.
Division with remainder of integers of length n can be done with $O(\mathsf{M}(n))$ word operations.

COROLLARY 9.9.
For an integer $m \in \mathbb{N}$ of length n, one multiplication in the residue class ring $\mathbb{Z}_m$ can be performed using $O(\mathsf{M}(n))$ word operations.

Algorithm 9.3 can be generalized to compute inverses modulo p^l for an element p in an arbitrary ring R. One minor difference is that we need an initial inverse modulo p, which is trivial to compute if $p = x$ and $R = D[x]$, and for a Euclidean domain R can be computed with the Extended Euclidean Algorithm in R.

ALGORITHM 9.10 p-adic inversion using Newton iteration.
Input: $f, g_0 \in R$ with $fg_0 \equiv 1 \bmod p$, where R is a ring (commutative, with 1) and $p \in R$ arbitrary, and $l \in \mathbb{N}$.
Output: $g \in R$ satisfying $fg \equiv 1 \bmod p^l$.

1. $r \longleftarrow \lceil \log l \rceil$
2. **for** $i = 1, \ldots, r$ compute $g_i \in R$ such that $g_i \equiv (2g_{i-1} - fg_{i-1}^2) \bmod p^{2^i}$
3. **return** g_r

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

EXAMPLE 9.11. We let $R = \mathbb{Z}$, and wish to compute the inverse of 5 modulo 81. We begin with $g_0 = -1$, since $-1 \cdot 5 \equiv 1 \bmod 3$. Then

$$g_1 \equiv 2g_0 - 5g_0^2 \equiv 2 \bmod 9, \quad g_2 \equiv 2g_1 - 5g_1^2 \equiv -16 \bmod 81,$$

and indeed $-16 \cdot 5 = -80 \equiv 1 \bmod 81$. $\diamond$

THEOREM 9.12.
Algorithm 9.10 correctly computes the inverse of $f \bmod p^l$. It uses $O(\mathsf{M}(l \log p))$ word operations if $R = \mathbb{Z}$, $p > 1$, and $|f| < p^l$, and $O(\mathsf{M}(l \deg p))$ operations in D if $R = D[x]$ for a (commutative) ring D, p is monic, and $\deg f < l \deg p$.

Exercise 9.8 asks for a proof.

In step 2, there are in general many g_i's satisfying the congruence. But in our applications such as integers or polynomials we have a unique "smallest" such g_i, namely from the standard system of representatives for $R/\langle p^{2^i} \rangle$ (see Section 25.2).

We have already seen in Section 4.2 that modular inverses can be computed using the Extended Euclidean Algorithm. If the modulus is a perfect power p^l with $l \in \mathbb{N}$ not too small and we use $\mathsf{M}(n) \in O(n \log n \operatorname{loglog} n)$, then the above theorem tells us that using the Extended Euclidean Algorithm to compute an inverse modulo p and then applying Algorithm 9.10 is asymptotically faster for our two main applications $R = \mathbb{Z}$ and $R = F[x]$, where F is a field, than using the Extended Euclidean Algorithm to compute the inverse modulo p^l. This is still true if we have an asymptotically fast Euclidean Algorithm like the one that will be discussed in Chapter 11.

Let R be a Euclidean domain, $p, f \in R$, and $l \in \mathbb{N}_{>0}$. Theorem 4.1 implies that

$$\begin{aligned} f \text{ is invertible modulo } p^l &\iff \gcd(p^l, f) = 1 \iff \gcd(p, f) = 1 \\ &\iff f \text{ is invertible modulo } p. \end{aligned}$$

(For $R = F[x]$ for a field F and $p = x$, this is equivalent to $f(0) \neq 0$, as we have seen before.) The following corollary says that this is true for arbitrary rings.

COROLLARY 9.13.
Let R be a ring (commutative, with 1), $p \in R$, and $l \in \mathbb{N}_{>0}$. An element $f \in R$ is invertible modulo p^l if and only if it is invertible modulo p.

PROOF. If f has an inverse modulo p, then Algorithm 9.10 computes an inverse of f modulo p^l. Conversely, if $g \in R$ is such that $fg \equiv 1 \bmod p^l$, then it follows from $l \geq 1$ that $fg \equiv 1 \bmod p$. $\square$

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

9.2. Generalized Taylor expansion and radix conversion

We now use fast division with remainder to compute quickly the ***p*-adic expansion** (or generalized Taylor expansion) of a polynomial a: given $a, p \in R[x]$ of degrees n, m, respectively, where R is a ring and p is monic, we want the unique $a_0, \ldots, a_{k-1} \in R[x]$ such that

$$a = \sum_{0 \le i < k} a_i p^i, \quad \deg a_i < m \text{ for } 0 \le i < k, \tag{4}$$

where $k = \lfloor n/m \rfloor + 1$ (see Section 5.11). This will be used in the integration algorithm of Chapter 22; the reader may skip the present section at first reading.

A special case is the usual coefficient sequence when $p = x$, or, more generally, the Taylor expansion of a around u for $p = x - u$ (Section 5.6). We have seen in Section 5.11 that the p-adic expansion can be computed using $O(n^2)$ operations in R, and we will now see how to do this in softly linear time.

ALGORITHM 9.14 Generalized Taylor expansion.
Input: $a, p \in R[x]$ with $\deg p = m$ and $\deg a < km$, where R is a ring (commutative, with 1), p is monic, and $k \in \mathbb{N}_{\ge 1}$ is a power of 2.
Output: $a_0, \ldots, a_{k-1} \in R[x]$ such that (4) holds.

1. **if** $k = 1$ **then return** $a_0 = a$
2. $t \longleftarrow k/2$
   call the repeated squaring algorithm 4.8 to compute $p^t \in R[x]$
3. $q \longleftarrow a \text{ quo } p^t, \quad r \longleftarrow a \text{ rem } p^t$
4. **call** the algorithm recursively to compute the generalized Taylor expansions $r = \sum_{0 \le i < t} a_i p^i$ and $q = \sum_{0 \le i < t} a_{t+i} p^i$
5. **return** $a_0, \ldots, a_{k-1}$

THEOREM 9.15.
Algorithm 9.14 correctly computes the p-adic expansion of a and uses at most $(3\mathsf{M}(km) + O(km)) \log k$ or $O(\mathsf{M}(km) \log k)$ operations in R.

PROOF. Correctness is clear when $k = 1$. If $k > 1$, then

$$a = qp^t + r = \sum_{0 \le i < t} a_{i+t} p^i p^t + \sum_{0 \le i < t} a_i p^i = \sum_{0 \le i < k} a_i p^i.$$

Let $T(k)$ denote the cost of the algorithm. Step 1 is for free, and hence $T(1) = 0$. By treating leading coefficients in polynomial multiplication separately, we have

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

$\mathsf{M}(n+1) \le \mathsf{M}(n) + 5n$ if $n \ge 1$ (Exercise 8.34). Thus the cost for step 2 is

$$\begin{aligned}
&\mathsf{M}(m+1) + \mathsf{M}(2m+1) + \cdots + \mathsf{M}\Big(\frac{km}{4}+1\Big) \\
&\quad \le \mathsf{M}\Big(m + 2m + \cdots + \frac{km}{4}\Big) + 5\Big(m + 2m + \cdots + \frac{km}{4}\Big) \\
&\quad \le \mathsf{M}\Big(\frac{km}{2}\Big) + \frac{5}{2}km \in \frac{1}{2}\mathsf{M}(km) + O(km).
\end{aligned}$$

The cost for step 3 is at most $5\mathsf{M}(km/2) + O(km)$ or $\frac{5}{2}\mathsf{M}(km) + O(km)$, by Theorem 9.6, and step 4 costs $2T(k/2)$. Thus we have the recursive inequality $T(k) \le 2T(k/2) + 3\mathsf{M}(km) + ckm$, for some constant $c \in \mathbb{R}$, and Lemma 8.2 implies that $T(k)$ is at most $(3\mathsf{M}(km) + O(km)) \log k$. □

One may also take $k = \lfloor (\deg f)/m \rfloor + 1$, not necessarily a power of 2, and $t = \lfloor k/2 \rfloor$ in step 2. The effect is a more "balanced" binary splitting in steps 3 and 4, possibly resulting in a slightly faster algorithm, but the analysis is more involved. If k is a power of 2, however, one may precompute $p^2, p^4, \ldots, p^{k/4}, p^{k/2}$, thus performing repeated squaring only once instead of every time the algorithm passes through step 2 in the recursive process.

Exercise 9.20 shows that the "reverse" task of computing the coefficients of a from its p-adic expansion (4) can also be done in time $O(\mathsf{M}(mk)\log k)$.

COROLLARY 9.16.
Let $n \in \mathbb{N}$ be a power of 2. The Taylor expansion of a polynomial $a \in R[x]$ of degree n around $u \in R$ can be computed using at most $(3\mathsf{M}(n) + O(n))\log n$ or $O(\mathsf{M}(n)\log n)$ operations in R.

The analog of Algorithm 9.14 for integers can be used to convert an integer from the 2^{64}-ary representation to an expansion with respect to the powers of an arbitrary base $p \in \mathbb{N}_{>1}$ in softly linear time. The following theorem about this **radix conversion** is proven in Exercise 9.21.

THEOREM 9.17.
Given $a, p \in \mathbb{N}$ with p of length m and a of length at most km for some $k, m \in \mathbb{N}$, we can compute the p-adic expansion of a using $O(\mathsf{M}(km)\log k)$ word operations.

9.3. Formal derivatives and Taylor expansion

In the next section, we solve the problem of finding a root g in a ring R of $\varphi(g) \equiv 0 \bmod p^l$, where $\varphi \in R[y]$ and $p \in R$. As in the case of inversion modulo p^l, Newton iteration will lead to a fast algorithm—provided we have a starting solution

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

modulo p. But first we need to adapt some well known tools from calculus to our purely algebraic setting.

DEFINITION 9.18. *Let R be an arbitrary ring (commutative, with 1). For $\varphi = \sum_{0\le i\le n}\varphi_i y^i \in R[y]$ we define the* ***formal derivative*** *of φ by*

$$\varphi' = \sum_{0\le i\le n} i\varphi_i y^{i-1}.$$

For $R=\mathbb{R}$, this is the familiar notion usually defined by a limit process. But in general, say over a finite field, there is no concept like a "limit". We note that i plays two different roles here: as a summation index, where it is really just a convenient notation for the vector $(\varphi_0,\dots,\varphi_n)\in R^{n+1}$ of coefficients, and the ring element $i = 1+\cdots+1 \in R$.

The formal derivative has some familiar properties.

LEMMA 9.19. *(i) $'$ is R-linear,*

(ii) $'$ satisfies the Leibniz (or product) rule $(\varphi\psi)' = \varphi'\psi + \psi'\varphi$,

(iii) $'$ satisfies the chain rule $(\varphi(\psi))' = \varphi'(\psi)\psi'$.

PROOF. (i) Let $\varphi,\psi\in R[y]$, $\varphi=\sum_{0\le i\le n}\varphi_i y^i$, $\psi=\sum_{0\le i\le n}\psi_i y^i$, and $a,b\in R$. Then

$$\begin{aligned}(a\varphi+b\psi)' &= (\sum_{0\le i\le n}(a\varphi_i+b\psi_i)y^i)' = \sum_{0\le i\le n} i(a\varphi_i+b\psi_i)y^{i-1}\\ &= a\sum_{0\le i\le n} i\varphi_i y^{i-1} + b\sum_{0\le i\le n} i\psi_i y^{i-1} = a\varphi' + b\psi'.\end{aligned}$$

(ii) Because of linearity, it is enough to show the claim for powers of y. So let $n,m\in\mathbb{N}$.

$$\begin{aligned}(y^n y^m)' &= (y^{n+m})' = (n+m)y^{n+m-1} = ny^{n-1}y^m + my^{m-1}y^n\\ &= (y^n)'y^m + (y^m)'y^n.\end{aligned}$$

(iii) Again, it is sufficient to show the claim for φ being a power of y, $\varphi = y^n$ for $n\in\mathbb{N}$ say. But then the claim reduces to $(\psi^n)' = n\psi^{n-1}\psi'$, which is easily proven using the Leibniz rule and induction on n. □

We note one difference from the usual derivatives, say over $\mathbb{R}$. Over $\mathbb{F}_p$ (or, more generally, any field of characteristic $p>0$) any pth derivative is zero. For example, $\varphi'' = 0$ for all $\varphi\in\mathbb{F}_2[y]$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

LEMMA 9.20. *Let $\varphi \in R[y]$ and $g \in R$. Then*

$$\varphi = \varphi(g) + \varphi'(g)(y-g) + \psi \cdot (y-g)^2$$

for some $\psi \in R[y]$.

PROOF. We have seen in Section 5.6 that φ has the Taylor expansion

$$\varphi = \sum_{0 \le i \le n} \varphi_i \cdot (y-g)^i = \varphi_0 + \varphi_1 \cdot (y-g) + \psi \cdot (y-g)^2,$$

around g, with unique $\varphi_i \in R$ and $\psi = \sum_{2 \le i \le n} \varphi_i (y-g)^{i-2} \in R[y]$. Substituting $y = g$ yields $\varphi_0 = \varphi(g)$. Now we take derivatives:

$$\varphi' = \varphi_1 + \psi' \cdot (y-g)^2 + \psi \cdot 2(y-g).$$

Again substituting $y = g$ yields $\varphi_1 = \varphi'(g)$, and the claim follows. □

This is the Taylor expansion of order 2 of φ around g. In fact, we can even consider g to be a new indeterminate, and then the lemma is true with $\psi \in R[y,g]$.

9.4. Solving polynomial equations via Newton iteration

Now we are ready to state the main result for the Newton iteration algorithm. Let R be an arbitrary ring and $p \in R$. The algorithm corresponding to (5) below computes better and better approximations, that is, modulo higher and higher powers m of p, to a root of a polynomial, just as the Newton iteration (3) does over the real numbers.

LEMMA 9.21 *(Quadratic convergence of Newton iteration). Let $m \in R$, $\varphi \in R[y]$, and $g, h \in R$ with $\varphi(g) \equiv 0 \bmod m$ and $\varphi'(g)$ invertible modulo m. Suppose that Newton's formula holds "approximately":*

$$h \equiv g - \varphi(g)\varphi'(g)^{-1} \bmod m^2. \tag{5}$$

Then $\varphi(h) \equiv 0 \bmod m^2$, $h \equiv g \bmod m$, and $\varphi'(h)$ is invertible modulo m^2.

Intuitively, if g is a "good" approximation to a zero of φ, then h is a "better" approximation, at least "twice as accurate".

PROOF. By Corollary 9.13, $\varphi'(g)$ is invertible modulo m^2, and hence the right hand side of (5) is well defined; Algorithm 9.10 computes $\varphi'(g)^{-1} \bmod m^2$ given $\varphi'(g)^{-1} \bmod m$. Since $m \mid m^2$, the congruence (5) also holds modulo m, and

$$h \equiv g - \varphi(g)\varphi'(g)^{-1} \equiv g \bmod m$$

because $\varphi(g)$ vanishes modulo m. This proves the second assertion.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

For the first one, we make use of the Taylor expansion given by Lemma 9.20 of φ around g and substitute h for y:

$$\begin{aligned}\varphi(h) &= \varphi(g)+\varphi'(g)(h-g)+\psi(h)\cdot(h-g)^2\\ &\equiv \varphi(g)+\varphi'(g)(h-g)\equiv\varphi(g)+\varphi'(g)\cdot(-\varphi(g)\varphi'(g)^{-1})\equiv 0 \bmod m^2.\end{aligned}$$

Here, we use the fact that m^2 divides $(-g)^2$, by the second assertion.

Since $h\equiv g \bmod m$, we have $\psi(h)\equiv\psi(g) \bmod m$ for *any* $\psi\in R[y]$, in particular for $\psi=\varphi'$. This is just a special case of a general principle: since the reduction map modulo m is a ring homomorphism, it commutes with the ring operations $+$ and $\cdot$, and hence with any polynomial over R. Now Corollary 9.13 proves the last claim. □

For a prime element m in a Euclidean domain R, the condition that $\varphi'(g)$ be invertible modulo m is equivalent to $\varphi'(g)\not\equiv 0 \bmod m$.

Note the similarities of the p-adic Newton iteration below to the Newton iteration for inversion (Algorithm 9.10).

ALGORITHM 9.22 p-adic Newton iteration.

Input: $\varphi\in R[y]$, where R is a ring (commutative, with 1), $p\in R$, $l\in\mathbb{N}_{>0}$, $g_0\in R$ a starting solution, with $\varphi(g_0)\equiv 0 \bmod p$ and $\varphi'(g_0)$ invertible modulo p, and a modular inverse s_0 of $\varphi'(g_0)$ modulo p.

Output: $g\in R$ with $\varphi(g)\equiv 0 \bmod p^l$ and $g\equiv g_0 \bmod p$.

1. $r\longleftarrow\lceil\log l\rceil$
2. **for** $i=1,\ldots,r-1$ compute $g_i,s_i\in R$ such that

$$g_i\equiv g_{i-1}-\varphi(g_{i-1})s_{i-1} \bmod p^{2^i},\quad s_i\equiv 2s_{i-1}-\varphi'(g_i)s_{i-1}^2 \bmod p^{2^i}$$

   { The second computation is the ith execution of step 2 in the Newton iteration of Algorithm 9.10 for the inversion of $\varphi'(g_i)$. }

3. compute $g\in R$ with $g\equiv g_{r-1}-\varphi(g_{r-1})s_{r-1} \bmod p^l$
   return g

THEOREM 9.23.

Algorithm 9.22 works correctly.

PROOF. Let $g_r\equiv g_{r-1}-\varphi(g_{r-1})s_{r-1} \bmod p^{2^r}$. Then $g\equiv g_r \bmod p^l$, and it is sufficient to show the invariants

$$g_i\equiv g_0 \bmod p,\quad \varphi(g_i)\equiv 0 \bmod p^{2^i},\quad s_i\equiv\varphi'(g_i)^{-1} \bmod p^{2^i} \text{ if } i<r$$

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

for $0 \le i \le r$ by induction on i. The case $i = 0$ is clear, and we assume that $i > 0$. Then the induction hypotheses imply that $p^{2^{i-1}}$ divides both $\varphi(g_{i-1})$ and $s_{i-1} - \varphi'(g_{i-1})^{-1}$, and hence p^{2^i} divides their product. Thus

$$g_i \equiv g_{i-1} - \varphi(g_{i-1})s_{i-1} \equiv g_{i-1} - \varphi(g_{i-1})\varphi'(g_{i-1})^{-1} \bmod p^{2^i},$$

and the first two invariants follow from Lemma 9.21 with $g = g_{i-1}$, $h = g_i$, and $m = p^{2^{i-1}}$. If $i < r$, then $g_i \equiv g_{i-1} \bmod p^{2^{i-1}}$ implies that

$$\varphi'(g_i)^{-1} \equiv \varphi'(g_{i-1})^{-1} \equiv s_{i-1} \bmod p^{2^{i-1}}.$$

Now s_i is obtained by one Newton step for inversion, as in Algorithm 9.3, and the third invariant follows. □

When $R = \mathbb{Z}$ or $R = F[x]$ for a field F and $p \in R$ is prime or irreducible, respectively, then finding a starting solution means computing a root of a polynomial in the field $K = R/\langle p \rangle$. We discuss this problem when K is finite or $K = \mathbb{Q}$ in Part III of this book.

EXAMPLE 9.24. (i) We take $R = \mathbb{Z}$ and $p = 5$, and determine a nontrivial solution (different from ± 1) of the equation $g^4 \equiv 1 \bmod 625$, so that $\varphi = y^4 - 1$. For a starting solution we can use $g_0 = 2$, since $\varphi(2) \equiv 0 \bmod 5$ (for example, by Fermat's little theorem) and $\varphi'(2) = 4 \cdot 2^3 \equiv 2 \not\equiv 0 \bmod 5$. Thus $s_0 \equiv 2^{-1} \equiv 3 \bmod 5$,

$$\begin{aligned} g_1 &\equiv g_0 - \varphi(g_0)s_0 = 2 - 15 \cdot 3 \equiv 7 \bmod 25, \\ s_1 &\equiv 2s_0 - \varphi'(g_1)s_0^2 = 2 \cdot 3 - 1\,372 \cdot 3^2 \equiv 8 \bmod 25, \\ g &\equiv g_1 - \varphi(g_1)s_1 \equiv 7 - 2\,400 \cdot 8 \equiv 182 \bmod 625, \end{aligned}$$

and indeed $182^4 = 1 + 1\,755\,519 \cdot 625$.

(ii) We take $R = \mathbb{F}_3[x]$ and $p = x$, and determine a square root g of the polynomial $f = x + 1$ modulo x^4 that satisfies $g(0) = -1$. Here $\varphi = y^2 - f \in \mathbb{F}_3[x][y]$, and $g_0 = -1$ can serve as a starting solution, since $g_0(0) = -1$, $\varphi(g_0) = -x \equiv 0 \bmod x$, and $\varphi'(g_0) = 2g_0 = 1 \not\equiv 0 \bmod x$. Thus $s_0 = 1$,

$$\begin{aligned} g_1 &\equiv g_0 - \varphi(g_0)s_0 = -1 - (-x) \cdot 1 = x - 1 \bmod x^2, \\ s_1 &\equiv 2s_0 - \varphi'(g_1)s_0^2 = 2 \cdot 1 - 2(x-1) \cdot 1^2 = x + 1, \\ g &\equiv g_1 - \varphi(g_1)s_1 = x - 1 - x^2(x+1) = -x^3 - x^2 + x - 1 \bmod x^4, \end{aligned}$$

and a calculation shows $(-x^3 - x^2 + x - 1)^2 = (x+1) + x^4(x^2 - x - 1)$. ◇

THEOREM 9.25.

When $R = D[x]$ for a ring D (commutative, with 1), $p = x$, $g_0 \in D$, $l \in \mathbb{N}$ is a power of 2, and $\varphi \in R[y]$ with $\deg_y \varphi = n$ and $\deg_x \varphi < l$, then Algorithm 9.22 takes $(3n + 3/2)\mathsf{M}(l) + O(nl)$ operations in D.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

PROOF. Reducing modulo x^{2^i} where possible, we may assume that the degrees of s_i and g_i are less than 2^i for all i. At first, we compute φ', taking $nl = n2^r$ operations in D. In step 2, we compute $\varphi(g_{i-1})$ and $\varphi'(g_i)$ modulo x^{2^i} using Horner's rule, at a total cost of $2n-1$ multiplications and the same number of additions modulo x^{2^i}, or $(2n-1)(\mathsf{M}(2^i)+2^i)$ operations in D. Computing g_i from g_{i-1}, s_{i-1}, and $\varphi(g_{i-1})$ can be done using at most $\mathsf{M}(2^{i-1})+2^{i-1} \leq \frac{1}{2}\mathsf{M}(2^i)+2^{i-1}$ operations in D: since the lower part of $\varphi(g_{i-1})$ is zero, we only need to multiply its upper part by s_{i-1} and take the negative of the lower part of the result as the upper part of g_i. Similarly, computing s_i from s_{i-1} and $\varphi'(g_i)$ takes $\mathsf{M}(2^i)+\mathsf{M}(2^{i-1})+2^{i-1} \leq \frac{3}{2}\mathsf{M}(2^i)+2^{i-1}$ operations, as in the proof of Theorem 9.4. Thus the cost for the ith iteration of step 2 is at most $(2n+1)\mathsf{M}(2^i)+2n\cdot 2^i$ operations in D, and similarly we have $(n+1/2)(\mathsf{M}(2^r)+2^r)$ operations in step 3. Now

$$\begin{aligned}\sum_{1\leq i<r}\left((2n+1)\mathsf{M}(2^i)+2n\cdot 2^i\right) &\leq \left((2n+1)\mathsf{M}(2^r)+2n\cdot 2^r\right)\sum_{1\leq i<r}2^{i-r}\\ &\leq (2n+1)\mathsf{M}(2^r)+2n\cdot 2^r,\end{aligned}$$

and the total cost is at most

$$(3n+\frac{3}{2})\mathsf{M}(2^r)+(4n+\frac{1}{2})2^r = (3n+\frac{3}{2})\mathsf{M}(l)+(4n+\frac{1}{2})l. \;\square$$

The more general statement when p is an arbitrary polynomial is discussed in Exercise 9.31, and Exercise 9.32 asks for a proof of the following integer analog.

THEOREM 9.26.
When $R=\mathbb{Z}$, $0 \leq g_0 < p$, and φ has degree n and coefficients absolutely less than p^l, then Algorithm 9.22 takes $O(n\,\mathsf{M}(l\log p))$ word operations.

When calculating by hand, one may perform all computations in Algorithm 9.22 in the p-adic representation, since then reductions modulo powers of p are for free.

One question that did not come up with the Newton iteration algorithm for inversion is that of uniqueness of the solution. Inverses modulo p^l are unique, but solutions of an arbitrary polynomial equation $\varphi(y)=0$ modulo p^l generally are not, because there may already be several solutions modulo p. The following theorem implies that for any $l \in \mathbb{N}_{>0}$, every starting solution gives rise to exactly one solution modulo p^l, so that there are as many solutions modulo p^l as there are modulo p (with nonvanishing φ').

THEOREM 9.27 *Uniqueness of Newton iteration.*
Let $\varphi \in R[y]$, $g \in R$ with $\varphi(g) \equiv 0 \bmod p$ and $\varphi'(g)$ invertible modulo p a starting solution, and $l \in \mathbb{N}_{>0}$. If $h, h^* \in R$ are solutions modulo p^l with $h \equiv g \equiv h^* \bmod p$ and $\varphi(h) \equiv 0 \equiv \varphi(h^*) \bmod p^l$, then $h \equiv h^* \bmod p^l$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

PROOF. Again, we make use of the Taylor expansion of φ and get

$$\varphi(h^*) = \varphi(h) + \varphi'(h)(h^* - h) + c \cdot (h^* - h)^2$$

for some $c \in R$, or equivalently

$$\varphi(h^*) - \varphi(h) = (h^* - h)(\varphi'(h) + c \cdot (h^* - h)). \tag{6}$$

Now

$$\begin{aligned} \varphi'(h) + c \cdot (h^* - h) &\equiv \varphi'(h) \bmod p \qquad (\text{since } h^* - h \equiv 0 \bmod p) \\ &\equiv \varphi'(g) \bmod p \qquad (\text{since } h \equiv g \bmod p). \end{aligned}$$

By Corollary 9.13, there exist some $s,t \in R$ such that $s \cdot (\varphi'(h) + c \cdot (h^* - h)) = 1 + tp^l$, and (6) implies that

$$s \cdot (\varphi(h^*) - \varphi(h)) - tp^l(h^* - h) = h^* - h.$$

The left hand side of this equation vanishes modulo p^l, and the claim follows. □

The conclusion of Theorem 9.27 need no longer be true if g violates the second condition for a starting solution, namely if $\varphi'(g)$ is not invertible modulo p. For example, the equation $y^4 = 0$ has only one solution $g \equiv 0$ modulo 5, but five solutions $h \equiv 0, 5, 10, 15, 20$ modulo 25 that are all congruent to 0 modulo 5. Here $\varphi = y^4$ and $\varphi'(0) \equiv 0 \bmod 5$, so that 0 is not a proper starting solution.

We will meet Newton iteration again in Chapter 15, under the name of Hensel lifting, which is used for (approximate) factorizations of polynomials.

9.5. Computing integer roots

In this section, we illustrate the prime power modular approach for solving problems over the integers with the example of computing roots of positive integers. We have introduced the concept in Chapter 5, but only now have the tool to fill in the details: Newton iteration.

Suppose that we are given positive $a, n \in \mathbb{N}$ and want to decide whether a is an nth power of an integer, and if so, we want to compute $\sqrt[n]{a} \in \mathbb{N}$. We might use numerical Newton iteration, applied to the equation $y^n - a = 0$, stop when the precision is sufficiently high, and then round to the nearest integer. A different approach is p-adic Newton iteration, where the precision management is somewhat easier.

For simplicity, we assume that n is odd (see Exercise 9.43 for square roots) and a is odd, by extracting the largest power of 2 dividing a and computing its nth root separately; if it does not have one, then a does not either. Then $1^n - a \equiv 0 \bmod 2$,

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

and $g_0 = 1$ is a valid starting solution for the 2-adic Newton iteration to solve $\varphi(y) = y^n - a = 0$, as in Algorithm 9.22, since $\varphi'(1) = n \cdot 1^{n-1} \equiv 1 \not\equiv 0 \bmod 2$. We choose $k \in \mathbb{N}$ minimal such that $2^{nk} > a$, and after $r = \lceil \log k \rceil$ steps Algorithm 9.22 has computed $g \in \mathbb{N}$ with $\varphi(g) = g^n - a \equiv 0 \bmod 2^k$. If now $g^n = a$ in $\mathbb{Z}$, then $g = \sqrt[n]{a}$. Otherwise, we claim that a is not an nth power in $\mathbb{Z}$. To see why, we assume that we have $b \in \mathbb{N}$ with $b^n = a$. Then b is odd, $b \equiv g_0 \equiv g \bmod 2$, $0 \le b < 2^k$, and

$$\varphi(b) = b^n - a = 0 \equiv g^n - a = \varphi(g) \bmod 2^k.$$

Now the uniqueness of Newton iteration (Theorem 9.27) yields $b \equiv g \bmod 2^k$, and since both sides are nonnegative and less than 2^k, they are equal.

In order to save computing time, we set $t_0 = 1$ in step 1 and additionally compute $t_i = g_i^{n-1} \operatorname{rem} 2^{2^{i+1}}$ in step 2. In the ith iteration of step 2 in Algorithm 9.22, we then calculate

$$g_i \equiv g_{i-1} - \varphi(g_{i-1})s_{i-1} \equiv g_{i-1} - (g_{i-1}t_{i-1} - a)s_{i-1} \bmod 2^{2^i}$$

with two multiplications and two additions modulo 2^{2^i}. Then we compute t_i and

$$s_i \equiv 2s_{i-1} - \varphi'(g_i)s_{i-1}^2 \equiv 2s_{i-1} - nt_i s_{i-1}^2 \bmod 2^{2^i},$$

taking three multiplications and two additions modulo 2^{2^i}. For the computation of t_i, we use repeated squaring (Section 4.3), at a cost of at most $2\log n$ multiplications modulo $2^{2^{i+1}}$. Thus the total cost for the ith iteration of step 2 is $O(\mathsf{M}(2^i)\log n)$ word operations. In the 2^{64}-ary representation, reduction modulo 2^{2^i} is essentially free.

THEOREM 9.28.
Let $a, n \in \mathbb{N}$ be odd, $a < 2^l$, and $3 \le n < l$. Then the above algorithm either computes the unique positive integer $\sqrt[n]{a} \in \mathbb{N}$, or certifies that a is not an nth power in $\mathbb{Z}$, using $O(\mathsf{M}(l))$ word operations.

PROOF. Correctness is clear from the above discussion. Let $c \in \mathbb{R}_{>0}$ such that the cost for the ith iteration of step 2 of the Newton iteration algorithm 9.22 is at most $c\,\mathsf{M}(2^i)\log n$ word operations and the cost for step 3 is at most $c\,\mathsf{M}(k)\log n$. With $r = \lceil \log k \rceil$ as above, the total cost is no more than

$$c\log n\Big(\mathsf{M}(k) + \sum_{1 \le i < r} \mathsf{M}(2^i)\Big) \le c\log n\left(\mathsf{M}(k) + \mathsf{M}\Big(\sum_{1 \le i < r} 2^i\Big)\right)$$

$$\le c\log n(\mathsf{M}(k) + 2\mathsf{M}(2^{r-1})) \le 3c\,\mathsf{M}(k)\log n.$$

Now $2^{n(k-1)} \le a < 2^l$, by the minimality of k, and hence $k - 1 < l/n$ and $\mathsf{M}(k) \in O(\mathsf{M}(\lfloor l/n \rfloor))$. Finally, we check whether $g^n = a$ using repeated squaring, at a cost of $O(\mathsf{M}(l))$ (Exercise 9.39), which dominates the cost for the other steps. □

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

EXAMPLE 9.29. Let us compute $\sqrt[3]{2197}$. We may choose $k = 4$, since $2^{3\cdot 4} = 2^{12} = 4096 > 2197$. Now $g_0 = s_0 = t_0 = 1$ and

$$\begin{aligned} g_1 &\equiv g_0 - (t_0 g_0 - 2197)s_0 \equiv 1 - (1-1) \equiv 1 \bmod 4, \\ t_1 &\equiv g_1^2 = 1 \bmod 16, \quad s_1 \equiv 2s_0 - 3t_1 s_0^2 = 2 - 3 \equiv 3 \bmod 4, \\ g &\equiv g_1 - (t_1 g_1 - 2197)s_1 = 1 - (1-5)\cdot 3 \equiv 13 \bmod 16, \end{aligned}$$

and in fact $2197 = 13^3$. $\diamond$

In Exercise 9.44, we use this to test whether $a \in \mathbb{N}_{>1}$ is a **perfect power**, so that $a = c^n$ for integers $c, n > 1$. In Sections 14.4 and 15.6, we discuss algorithms for computing integer roots of arbitrary polynomials.

9.6. Newton iteration, Julia sets, and fractals

This section gives a general framework for Newton iteration, and presents some of the similarities and differences with the method over the real numbers. These results are not needed later. The real (or complex) iteration makes basic use of the notion of convergence. This can be carried over to numbers and polynomials, by saying that two elements are close together if their difference is divided by a large power of our prime p.

DEFINITION 9.30. *A **valuation** on an integral domain R is a map $v: R \longrightarrow \mathbb{R}$ which is multiplicative, subadditive, and positive definite, so that it satisfies for all $a, b \in R$*

(i) $v(ab) = v(a)v(b)$,

(ii) $v(a+b) \leq v(a) + v(b)$,

(iii) $v(a) \geq 0$ *and* $v(a) = 0$ *if and only if* $a = 0$.

The following are some commonly used valuations on integers and polynomials. A ring may, of course, have more than one valuation.

EXAMPLE 9.31. (i) Let $R = \mathbb{Z}$, and $v(a) = |a|$, the **absolute value**.

(ii) With $R = \mathbb{Z}$ and p prime, let

$$v_p(a) = \begin{cases} 0, & \text{if } a = 0, \\ p^{-n}, & \text{if } p^n \mid a \text{ and } p^{n+1} \nmid a. \end{cases}$$

This is the ***p*-adic valuation**.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

(iii) With $R = F[x]$, F a field, let

$$v(a) = \begin{cases} 0, & \text{if } a = 0, \\ 2^{-n}, & \text{if } x^n \mid a \text{ and } x^{n+1} \nmid a. \end{cases} \tag{7}$$

This is the ***x*-adic valuation**. Similarly, if $p \in F[x]$ is irreducible, we get the p-adic valuation if x is replaced by p in (7).

(iv) With $R = F[x]$, F a field, let

$$v(a) = \begin{cases} 0, & \text{if } a = 0, \\ 2^{\deg(a)}, & \text{if } a \neq 0. \end{cases}$$

This is the **degree valuation**. $\diamond$

A similar p-adic valuation can be defined for any prime p in a UFD.

The notion of being "close together" can be expressed in terms of a valuation. In the polynomial case with valuation (iii), we use the intuition that a polynomial a is small if $x^n \mid a$ for some large n. Two polynomials a, b are close if their distance $d(a,b) = v(a-b)$ is small.

EXAMPLE 9.32. $v_3(54) = 3^{-3} = \dfrac{1}{27}, \quad v_3(55) = 1, \quad v_3(54000000) = \dfrac{1}{27}$. $\diamond$

DEFINITION 9.33. *A **non-Archimedean valuation** is a valuation with the property of subadditivity replaced by the stronger condition that for all $a, b \in R$*

$$v(a+b) \leq \max\{v(a), v(b)\}.$$

*This is the **ultrametric inequality**.*

The p-adic valuation for integers is non-Archimedean, while the absolute value for integers is Archimedean. The Newton iteration of Algorithm 9.22 for solving polynomial equations approximately can be carried over to any non-Archimedean valuation. Lemma 9.21 reads as follows in this generality.

LEMMA 9.34. *Let v be a non-Archimedean valuation on R, with $v(a) \leq 1$ for all $a \in R, \varphi \in R[y], 0 < \varepsilon < 1$, and $g, h \in R$ with $v(\varphi(g)) \leq \varepsilon$ and $v(\varphi'(g)) = 1$, and suppose that*

$$v(h - (g - \varphi(g)/\varphi'(g))) \leq \varepsilon^2.$$

Then $v(\varphi(h)) \leq \varepsilon^2$, $v(h-g) \leq \varepsilon$, and $v(\varphi'(h)) = 1$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

PROOF. We only show the first two bounds, using the Taylor expansion (Lemma 9.20) of φ around g:

$$\begin{aligned}
v(h-g) &= v\Big(h-g+\frac{\varphi(g)}{\varphi'(g)}-\frac{\varphi(g)}{\varphi'(g)}\Big) \le \max\left\{v\Big(h-g+\frac{\varphi(g)}{\varphi'(g)}\Big), v\Big(\frac{\varphi(g)}{\varphi'(g)}\Big)\right\} \\
&= \max\{\varepsilon^2,\varepsilon\} = \varepsilon, \\
v(\varphi(h)) &= v(\varphi(g)+\varphi'(g)(h-g)+\psi(h-g)\cdot(h-g)^2) \\
&= v\Big(\varphi(g)-\varphi'(g)\frac{\varphi(g)}{\varphi'(g)}+\varphi'(g)\Big(h-g+\frac{\varphi(g)}{\varphi'(g)}\Big)+\psi(h-g)\cdot(h-g)^2\Big) \\
&\le \max\left\{v(\varphi'(g))\cdot v\Big(h-g+\frac{\varphi(g)}{\varphi'(g)}\Big), v(\psi(h-g))\cdot v(h-g)^2\right\} \\
&\le \max\{1\cdot\varepsilon^2, 1\cdot\varepsilon^2\} = \varepsilon^2. \ \square
\end{aligned}$$

The division by $\varphi'(g)$ in the above formulas leads, in principle, out of the ring R. There are three ways of dealing with this problem: we can replace $\varphi'(g)^{-1}$ by a sufficiently good approximation in R, as computed by Algorithm 9.10, or we can extend v to the field of fractions of R, by setting $v(a/b) = v(a)/v(b)$ if $b \neq 0$, or we can multiply by $\varphi'(g)$, where necessary, and conclude from $v(\varphi'(g)) = 1$ that the valuations do not change.

Newton iteration in $R = \mathbb{Q}$ for solving $y^2 - 2 = 0$ and starting with $g_0 = 2$ leads to better and better rational approximations to the root. But $\sqrt{2}$ itself is not a rational number; in order to capture such an exact root, the domain has to be enlarged, say to $\mathbb{R}$.

A similar phenomenon happens with the p-adic valuations on $\mathbb{Z}$ or $F[x]$. One can enlarge these rings to their **completions**, namely to the ring $\mathbb{Z}_{(p)}$ of p-adic integers or the ring $F[[x]]$ of formal power series (for the x-adic valuation), and in these larger rings Newton iteration converges to an exact root. We do not go into details, since these quantities cannot be represented in a finite manner, and these rings are mainly of conceptual interest to computer algebra. One can finitely represent initial segments of them, say $a \bmod p^l$ for $a \in \mathbb{Z}_{(p)}$, but that is essentially the same as some integer modulo p^l.

Newton iteration for inversion tells us what the units are in these rings. An element $a = a_0 + a_1 p + a_2 p^2 + \cdots \in \mathbb{Z}_{(p)}$, with $a_0, a_1, a_2, \ldots \in \{0, \ldots, p-1\}$, is a unit if and only if $a_0 \bmod p$ is a unit in $\mathbb{Z}_p$, that is, if and only if $a_0 \neq 0$. The power series $a = a_0 + a_1 x + a_2 x^2 + \cdots \in F[[x]]$, with $a_0, a_1, a_2, \ldots \in F$ is a unit in $F[[x]]$ if and only if $a_0 \neq 0$. As an example, $1 - x \in F[[x]]$ is a unit, with inverse $1 + x + x^2 + \cdots$.

In the real or complex numbers, the behavior of Newton iteration is quite intricate, as seen in the simple case of finding the three roots $1, e^{2\pi i/3}, e^{4\pi i/3}$ in $\mathbb{C}$ of $\varphi = y^3 - 1$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

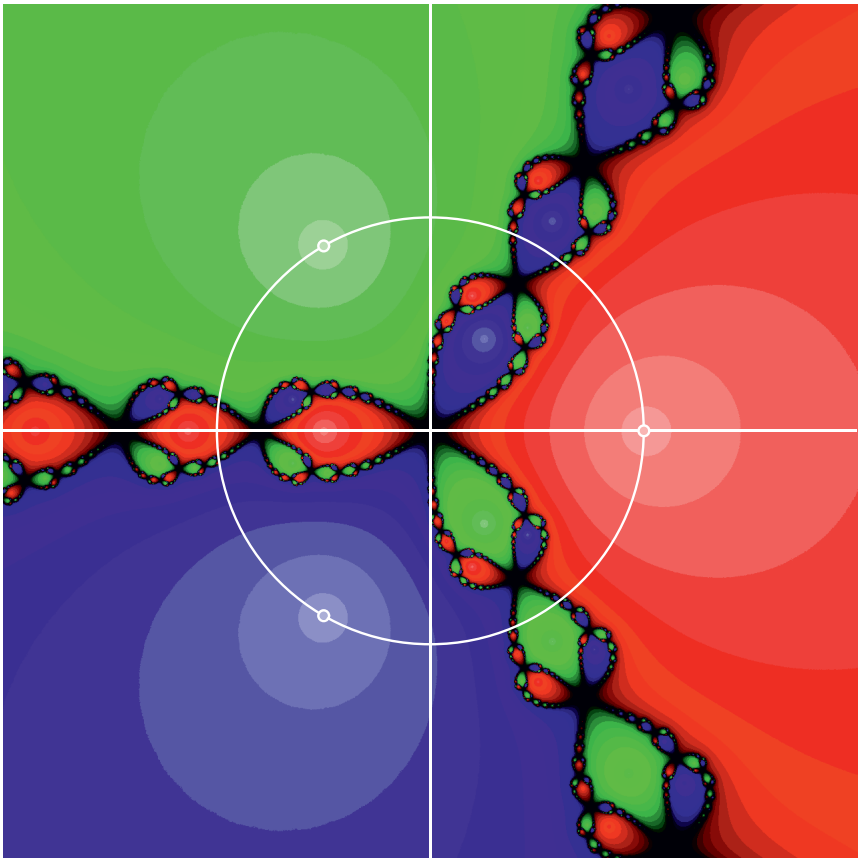


FIGURE 9.2: Convergence of Newton iteration to solve $y^3 = 1$ over $\mathbb{C}$.

In Figure 9.2, the three roots are marked by white circles, and the colored areas indicate convergence to one of them via the Newton iteration. Brightness corresponds to the "convergence speed": the brighter the color of a point, the earlier does Newton iteration starting at that point approach its final limit. The intricacy of the picture illustrates the difficulty of finding a simple rule for telling where a point will go. We have big bright areas where the limit point is clear, but also other areas where a small change of the initial value will lead to a different destination. The points on the real line have nowhere to go but to 1. But the real root of $16x^9 + 51x^6 + 21x^3 + 2$ near -1.43 first crashes into 0 and then explodes ...

This problem is part of a larger question: given an iteration function $g_{i+1} = \psi(g_i)$, determine the behavior for any starting value g_0. For example, for which g_0 does this converge at all? The set of all these g_0 is called the **Julia set** of ψ, after the French mathematician Gaston Julia who first studied it. Such sets are highly complicated and provide stunning pictures. Their study is mainly a part of *dynamical systems theory*. The beautiful mathematical theory of chaos and fractals

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

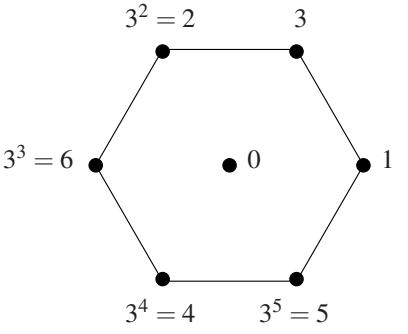


FIGURE 9.3: Representation of $\mathbb{Z}_7$ in Figure 9.4.

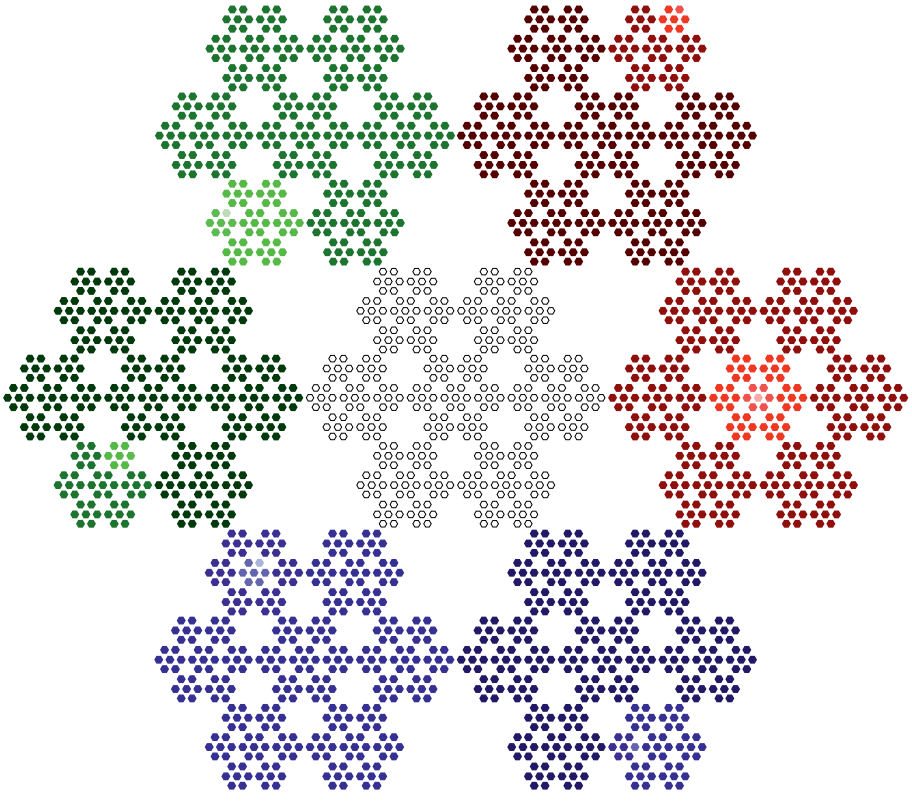


FIGURE 9.4: Convergence of Newton iteration to solve $y^3 = 1$ over the 7-adic integers.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

is described and richly illustrated in Mandelbrot (1977) and Peitgen, Jürgens & Saupe (1992).

In Figure 9.4, we see the analog of Figure 9.2 over $\mathbb{Z}_{(7)}$, the set of 7-adic integers. The seven elements of $\mathbb{Z}_7$ are arranged as in Figure 9.3, and $\mathbb{Z}_{(7)}$ can be represented by the fractal composed of infinitely many recursive compositions of this centered hexagon. The boundary is a **Koch snowflake**. The derivative $\varphi' = 3y^2$ of $\varphi = y^3 - 1$ vanishes modulo 7 on the white points in the center, and Newton iteration does not work. All other points converge to the root of f of their color, and brighter color means faster convergence. The three roots of φ in $\mathbb{Z}_{(7)}$, whose sum equals 0, are $1, 2+4\cdot 7+6\cdot 7^2+3\cdot 7^3+\cdots$, and $4+2\cdot 7+0\cdot 7^2+3\cdot 7^3+\cdots$. In $\mathbb{Z}_7^\times$, at most one iteration leads to a root modulo 7 and then to convergence modulo higher powers of 7. However, for large p, it seems to be as tricky as in the complex situation to determine which elements of $\mathbb{Z}_p^\times$ lead to a root modulo p, except that we have only a finite number of starting points to try. Once a root is reached, convergence is quick.

9.7. Implementations of fast arithmetic

In this book you can learn some of the fundamental algorithms in computer algebra, with an emphasis on the modern fast methods that one needs to solve large problems. The enterprising reader might now say to herself: "Ok, so let's do it. Let's build a computer algebra system." But that is a tall order. Not only does it require an enormous effort of dozens (or hundreds) of woman-years, but it also needs many tools that cannot even be mentioned in this book. Some of them become apparent as soon as you start up your favorite system on your own machine.

Nevertheless, we present in this section some notes on implementations, mainly a case study of two software packages, one designed by Victor Shoup and the other one by the authors. These packages are rather limited in scope; Shoup's basically provides fast arithmetic for polynomials and matrices over finite fields and the integers, and ours only works over $\mathbb{F}_2$. The fact that even the latter modest goal requires about 10000 lines of code indicates the amount of effort necessary to implement carefully the basic algorithms.

The framework for determining the (arithmetic) cost of our algorithms is *asymptotic analysis*, with a typical statement like "$O(n\log n\operatorname{loglog} n)$ operations" for multiplication. This is a powerful, reliable, and universal tool for comparing algorithms. For any new algorithm, an improvement of the previously known asymptotic estimates is sufficient justification of its relevance; however, such improvements are hard to come by for well-studied problems. This section describes some of the additional efforts that have to supplement the asymptotic analysis for any practical implementation.

By its nature, the flavor of this section is rather different from the rest of the book (except that in Section 15.7 we continue this report on implementations).

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

Many of our algorithms are (hopefully) of long-lasting interest, but the computer timings reported now will be out of date before the book goes to press.

The first lesson in implementing a software package for fast integer or polynomial arithmetic is that a large variety of algorithms have to be coded and tested to determine the **crossover points**. These are the input sizes at which one algorithm beats another one. A typical experience is that, say for multiplication, the classical method is best for small inputs, Karatsuba's algorithm takes over for intermediate sizes, and a fast, for example, an FFT-based method, excels for large problems.

The second lesson is that just casting the algorithms "from the book" into software will not work well. One has to understand the algorithmic ideas in depth and use a multitude of tricks and special relations to make things go at lightning speed. Only a few of these methods can be explained here; fortunately, there is no limit to the ingenuity of the programmer (except for having to complete the project in some reasonable time frame).

Several factors determine whether a software package for (integer or polynomial) arithmetic is fast in practice. Besides choosing the algorithms and determining the crossover points between various methods, one has to design suitable data types, exploit fast hardware arithmetic whenever possible, and customize for specific types and sizes of problems.

Currently, there are—besides implementations in any general purpose computer algebra system—several libraries available for arbitrary precision integer arithmetic and univariate polynomial arithmetic over finite fields, $\mathbb{Z}$, $\mathbb{Q}$, algebraic number fields, $\mathbb{R}$, and $\mathbb{C}$ (among others GNU MP, PARI, LIDIA), but only few that implement the fast algorithms presented in Chapters 8 through 11. Among them are LIP by Arjen Lenstra and Paul Leyland, the package of Schönhage, Grotefeld & Vetter (1994) (see also Reischert 1995), NTL by Shoup (an early version is described in Shoup 1995), and BIPOLAR (Binary Polynomial Arithmetic) by von zur Gathen & Gerhard (1996). The last two of these will be described below.

The C++ library BIPOLAR was designed and optimized for univariate polynomial factorization over $\mathbb{F}_2$. This is a very narrow focus, but we use it to explain some general principles. The first question when writing the package was the choice of *data types*. When programming on top of an existing package, one may not have much choice. Experience has shown that for high performance code, one should represent algebraic data as compactly as possible, since all linear operations like addition or copying take time proportional to the length of the representation (that is, the number of machine words it occupies in memory). Thus, on a machine with a word size of 32 bit, we represent polynomials over $\mathbb{F}_2$ as arrays of 32 bit words; each word contains 32 consecutive coefficients. In this representation, all linear-time operations are straightforward to implement, and the next task is to tackle the nontrivial arithmetic operations, starting with multiplication.

We have five methods at our disposal:

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

- table lookup,
- classical multiplication,
- Karatsuba's algorithm,
- an algorithm by Cantor (1989),
- FFT-based algorithms.

We did not experiment with the last one. As explained above, each method typically has its range of input sizes where it beats the other methods. One has to implement many variations of these approaches and test them to determine the best one for each range, starting with the small ranges. A typical outcome then is a **hybrid** algorithm where one performs, say, first a few Karatsuba steps and then classical multiplication on small arguments. As an example, for **single precision** polynomials of degree less than 32 we found the following to work best: two stages of Karatsuba's algorithm plus table lookup for the 9 resulting multiplications at degrees less than 8; the size of the table is $2^8 \cdot 2^8 \cdot 16$ bits or 128 kilobytes. (Unfortunately, there is no hardware support for multiplication in $\mathbb{F}_2[x]$ on general purpose microprocessors—there is no possibility to sever the "carry" line—and one has to implement the single precision multiplication in software.) On top of this, both the classical algorithm 2.3 and Karatsuba's algorithm 8.1 are implemented at machine word level, that is, with base x^{32} instead of x; the block sizes are multiples of 32 and recursion in the latter algorithm stops as soon as the polynomials are of degree less than 32.

We also implemented an algorithm by Cantor (1989) for multiplication in $\mathbb{F}_2[x]$, which uses evaluation and interpolation at linear subspaces of $\mathbb{F}_{2^m}$ for some $m \in \mathbb{N}$ and is similar to the FFT based methods from Chapter 8. Its running time is $O(n(\log n)^{1.59})$ arithmetic operations in $\mathbb{F}_2$. For practical purposes, we may take $m = 32$, so that one element of $\mathbb{F}_{2^m}$ fits precisely into a machine word. Here the single precision operations are multiplications in $\mathbb{F}_{2^{16}}$ and $\mathbb{F}_{2^{32}}$. Again, we might have implemented these by doing one polynomial multiplication in the way described above and one subsequent division with remainder, but we have chosen a different approach using tables based on the multiplicative structure of finite fields, as in Pollard (1971) and Montgomery (1991). We take a fixed generator g of the multiplicative group $\mathbb{F}_{2^{16}}^{\times}$, and compute two tables for the exponentiation map $\{0,\ldots,2^{16}-2\} \longrightarrow \mathbb{F}_{2^{16}}^{\times}$, with $a \longmapsto g^a$, and its inverse. Two nonzero elements $c,d \in \mathbb{F}_{2^{16}}$ are multiplied by determining $a,b \in \{0,\ldots,2^{16}-2\}$ such that $c = g^a$ and $d = g^b$, and computing $cd = g^{a+b}$. This amounts to essentially one addition modulo $2^{16}-1$ and three table lookups; the size of each table is $2^{16} \cdot 16$ bits or 128 KB. Inversion in $\mathbb{F}_{2^{16}}^{\times}$ is done similarly. One multiplication in $\mathbb{F}_{2^{32}}$ is reduced to three multiplications in $\mathbb{F}_{2^{16}}$ à la Karatsuba; this requires a change of basis.

After determining the best (that is, the fastest) routines for single precision arithmetic, we implemented the three multiplication algorithms mentioned above for

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

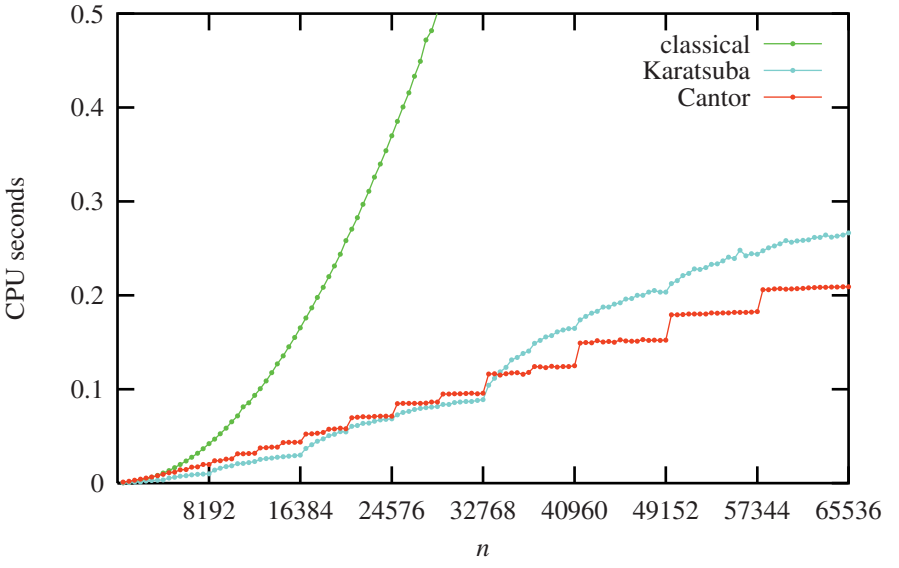


FIGURE 9.5: Multiplication of polynomials in $\mathbb{F}_2[x]$ of degree $n-1$ in BIPOLAR.

multiprecision polynomials. An important aspect here is space considerations: due to the overhead involved in dynamic storage management, one should try to limit dynamic allocation whenever possible. When starting with two polynomials of length at most 2^k machine words each, one can implement Karatsuba's algorithm so as to use at most 2^{k+2} words of work space. In our implementation, we pre-allocate this amount of storage and free it again at the end of the computation. Both the multipoint evaluation and the interpolation in Cantor's algorithm can be implemented **in place**, which means that the output is returned in the space of the input, and no additional work space (besides a constant number of registers) is required (the same is true for the FFT; see Figure 8.4). Figure 9.5 gives running times for the three multiplication algorithms. The experiments in this section ran in 1998 on a Sparc Ultra 1 with 167 MHz. The timings shown are the average of 10 pseudorandom inputs; the relative standard deviation was less than 10% (except in some cases where the time was less than 0.01 seconds).

The recursive algorithms work particularly well at degrees just below a power of 2, and a direct implementation gives a picture with large steps at these powers. In Figure 9.5 you can see the effect of several tricks that have made the behavior of Karatsuba's algorithm quite smooth, and for Cantor's algorithm have broken the big step between $2^{15}-1$ and 2^{15} (to $2^{16}-1$) into four smaller steps.

The crossover points between the algorithms are near degree 500 for Karatsuba's algorithm, and near degree 33 000 for Cantor's algorithm (see Figure 9.5). After determining these, we have built a hybrid multiplication algorithm which compares

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

the degrees of the input polynomials to the two crossover degrees and then decides which of the three algorithms to use. Its performance can be seen in Table 9.6.

n	CPU seconds
512	0.0004
1024	0.0006
2048	0.0014
4096	0.0038
8192	0.0110
16 384	0.0329
32 768	0.0971
65 536	0.2135
131 072	0.4666
262 144	1.0218
524 288	2.2330
1 048 576	4.9560

TABLE 9.6: Multiplication of polynomials in $\mathbb{F}_2[x]$ of degree $n-1$ in BIPOLAR using the hybrid algorithm.

We have to stress that here (and in all our implementation discussions) the timings and crossover points depend on *our* efforts and on *our* computing environment. We expect our software to perform quite well on other similar processors, but for example to use the power of true 64 bit machines one would have to start all over again—at least for the single precision arithmetic. The one universal truth is that a well-done implementation is very labor-intensive and requires close familiarity with the algorithmics.

For division with remainder in $\mathbb{F}_2[x]$, we first wrote single precision routines for both the classical algorithm and Newton inversion, working at the bit level. On top of these, we implemented multiprecision versions working at machine word level (Algorithm 2.5 with base x^{32} instead of x, and Algorithm 9.3 followed by essentially two polynomial multiplications). The asymptotically fast division algorithm uses the hybrid multiplication algorithm as a subroutine. Figure 9.7 shows some experiments; the crossover point between the two algorithms (the top two curves) is near degree 10000.

In some applications, in particular in modular arithmetic and polynomial factorization, many remainder computations modulo a fixed divisor $f \in \mathbb{F}_2[x]$ have to be performed. In that case, $\mathrm{rev}(f)^{-1} \bmod x^{\deg f}$ can be precomputed using Algorithm 9.3 and stored, and then one remainder computation amounts to essentially two polynomial multiplications of degree about $\deg f$. When counting only the latter, the crossover point drops to about 4000 (Figure 9.7). Further optimization is possible when $\deg f$ is above the crossover degree for Cantor multiplication, which reduces the time for one remainder computation modulo f to essentially the time for a polynomial multiplication of the same size. Similar optimizations are

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

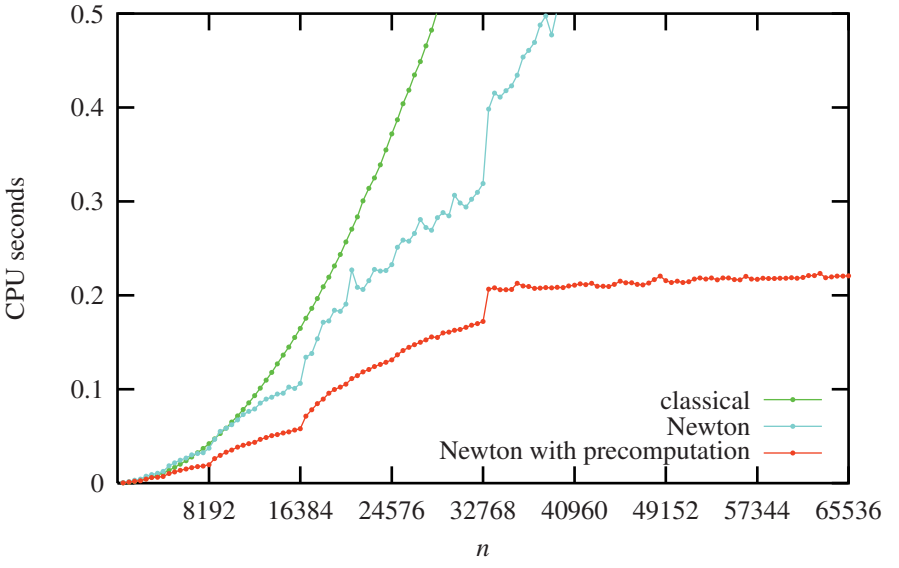


FIGURE 9.7: Division of a polynomial in $\mathbb{F}_2[x]$ of degree $2n-2$ by a polynomial of degree $n-1$ with remainder in BIPOLAR.

possible when many modular multiplications gh rem f with both f and h fixed are performed.

BIPOLAR also implements the Extended Euclidean Algorithm and polynomial factorization routines for $\mathbb{F}_2[x]$; the latter will be discussed in Section 15.7.

The integer arithmetic of Shoup's NTL is highly optimized. On a processor with a word length of 32 bits, arbitrary precision integers are represented as arrays of machine words, where—depending on the underlying hardware—between 26 and 30 consecutive bits of the binary representation are packed into one machine word. Multiplication and division of such single precision integers is done by cleverly employing the hardware floating point arithmetic, which in most currently available microprocessors is considerably faster than hardware integer arithmetic.

NTL uses classical integer multiplication for integers of size up to about 500 bits, and Karatsuba's algorithm for larger integers. Other arithmetic operations like division with remainder and the Extended Euclidean Algorithm are all done in the classical way.

We have implemented the classical algorithm 2.4, Karatsuba's algorithm 8.1, the three primes FFT algorithm 8.25, and the algorithm of Schönhage & Strassen (1971) for integer multiplication using low-level routines of NTL version 1.5. Figure 9.8 gives running times for our implementations and the built-in routine of NTL. We have not invested much effort into optimizing our routines, and the timings of our Karatsuba implementation are at most twice as large as those of NTL's routine. The graphs for the algorithms which are not FFT-based are quite smooth,

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

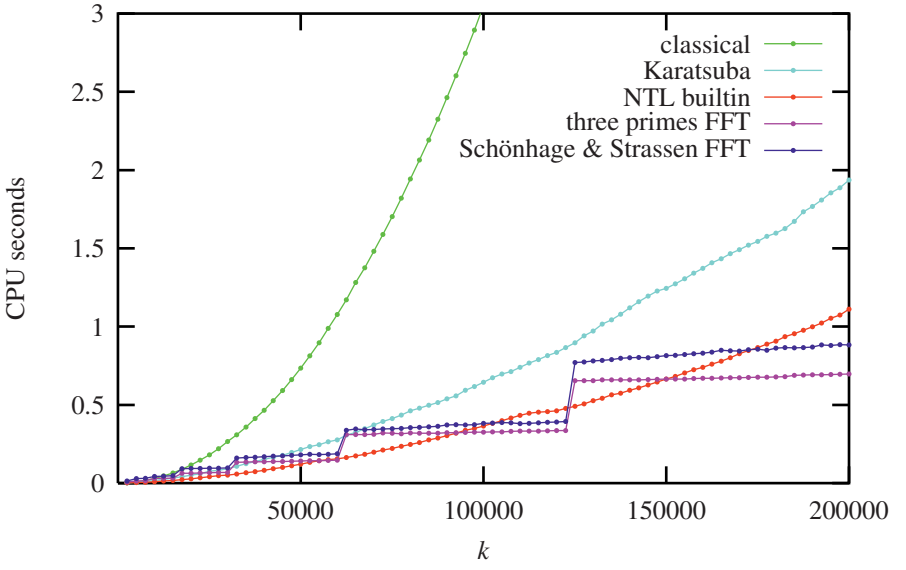


FIGURE 9.8: Multiplication of k-bit integers in NTL.

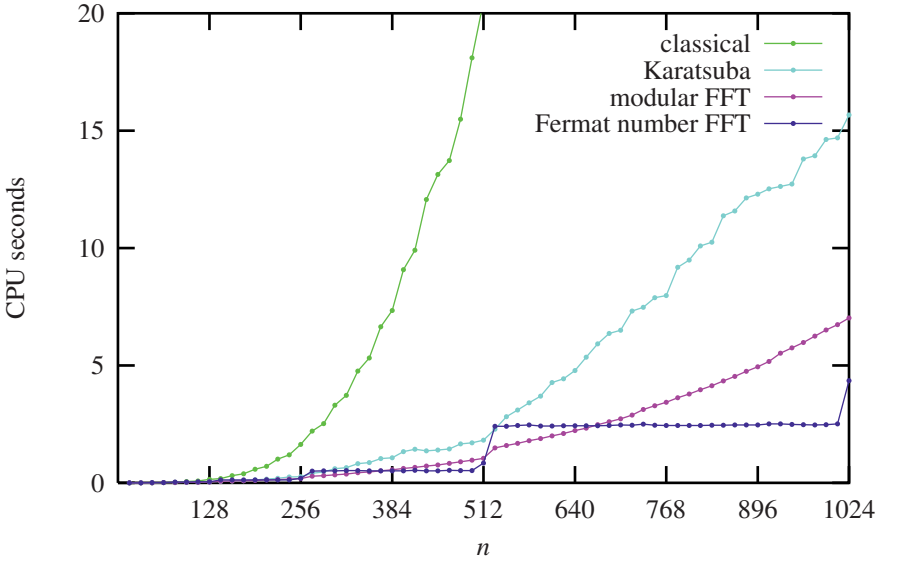


FIGURE 9.9: Multiplication of polynomials of degree $n-1$ with n bit integer coefficients in NTL.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

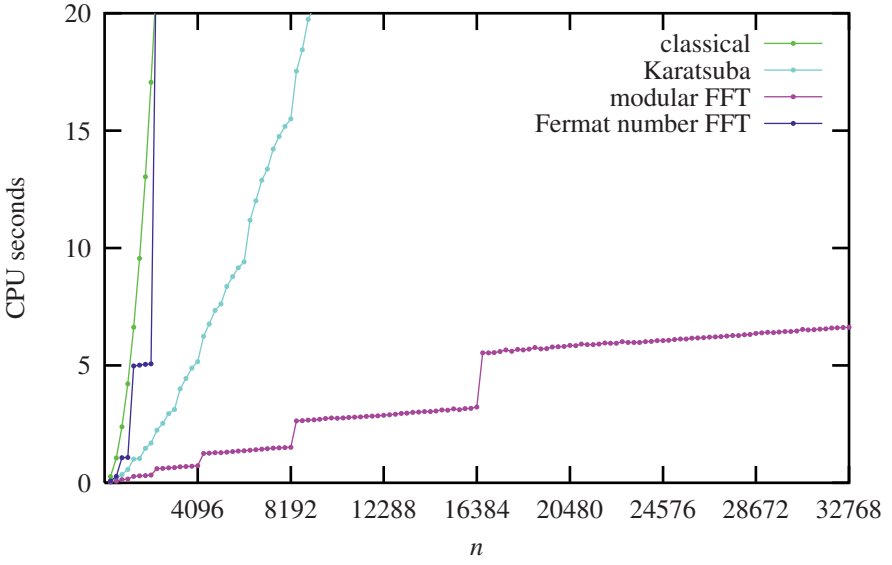


FIGURE 9.10: Multiplication of polynomials of degree $n-1$ with 64 bit integer coefficients in NTL.

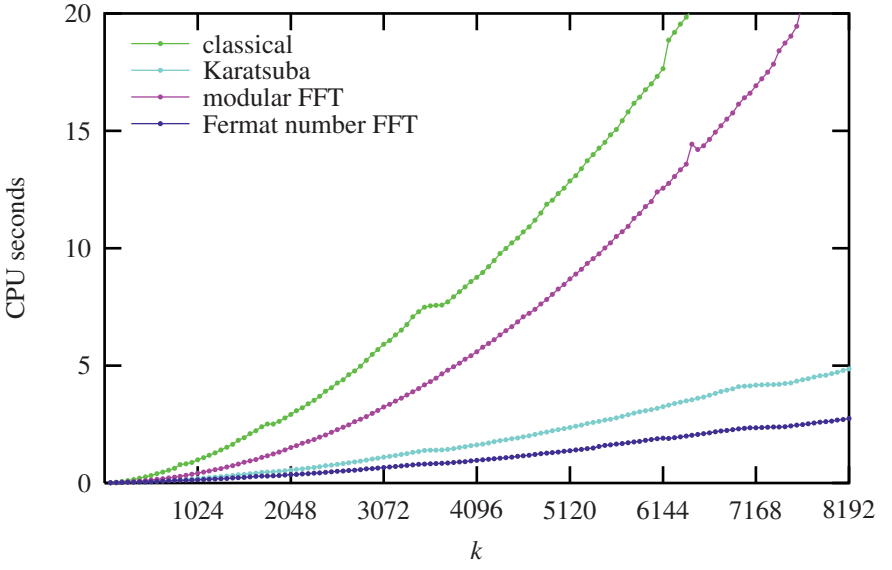


FIGURE 9.11: Multiplication of polynomials of degree 63 with k bit integer coefficients in NTL.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

while we have large steps near powers of 2 for the three primes FFT and Schönhage & Strassen's algorithm. These steps may be smoothed with some additional effort, but we have not tried this.

For multiplying polynomials over $\mathbb{Z}$ and $\mathbb{Z}_m$ with $m \in \mathbb{Z}$, NTL implements the classical algorithm for small degrees and coefficient sizes, Karatsuba's algorithm for polynomials of medium degree, and the FFT-based modular approach described in Section 8.4 and a variant of Algorithm 8.20 using FFT modulo Fermat numbers (Exercise 8.36) for larger polynomials. Figures 9.9 through 9.11 show running times for various degrees and coefficient sizes in NTL. For division with remainder, NTL uses the classical algorithm for polynomials of small degree, and Newton inversion (Algorithm 9.3) for higher degree polynomials.

Besides basic arithmetic for multiprecision integers, floating point numbers, finite fields, and univariate polynomials and matrices over these domains, the recent version 3.1 of NTL includes routines for primality testing (Chapter 18), Chinese remaindering (Chapters 5 and 10), computing greatest common divisors (Chapters 3 and 6), factorization of univariate polynomials (Part III), computing reduced bases in lattices over $\mathbb{Z}$ (Chapter 16), and much more. The polynomial factorization routines will be discussed in Section 15.7. NTL is a `C++` library and can be downloaded from Victor Shoup's homepage `http://www.shoup.net`. We recommend this package to anybody who is not ready to reinvent the wheel.

Notes. **9.1.** Cook (1966) devised a division algorithm for integers that costs the same number of word operations as a multiplication, up to a constant factor. Sieveking (1972), Strassen (1973a), Kung (1974), and Borodin & Moenck (1974) gave analogous algorithms for polynomials. For the details of the division method for integers, see Knuth (1998), Algorithm 4.3.3 R, and Aho, Hopcroft & Ullman (1974), §8.2. In the nonscalar model, Schönhage has shown how to divide a polynomial of degree at most $2n$ by one of degree n with $5.875n$ multiplications and divisions (see Kalorkoti 1993 and Bürgisser, Clausen & Shokrollahi 1997, Corollary 2.26 and Notes 2.8). Karp & Markstein (1997) state a modification of Algorithm 9.5 taking only $\frac{7}{2}\mathsf{M}(m) + \mathsf{M}(n) + O(n)$ ring operations. Burnikel & Ziegler (1998) give a divide-and-conquer algorithm for division with remainder taking time about $2\mathsf{M}(n)$ when Karatsuba multiplication is used; see also Jebelean (1997).

9.2. Algorithm 9.14 is from von zur Gathen (1990a). In fact, the Taylor expansion can be computed with $\mathsf{M}(n) + O(n)$ or $O(\mathsf{M}(n))$ ring operations (Aho, Steiglitz & Ullman 1975, see also Schönhage, Grotefeld & Vetter 1994, page 284, and Exercise 9.49).

9.3. The Taylor expansion goes back to Taylor (1715) and Maclaurin (1742), and is already in Newton (1710) for $\varphi = y^n$.

9.4 and 9.5. The formulas of Newton's iteration for square and cube roots were known by the Babylonians, and appear in the 6th century Indian text Āryabhaṭīya. Muḥammad al-Khwārizmī described the Newton iteration for square roots around 830 (see Folkerts 1997). Jamshīd Al-Kāshī, who lived in Samarkand in the early 15th century, had used a single Newton step for root finding. Both one- and two-dimensional Newton iteration is explicitly described in Waring (1770). The history of Newton's method is traced in Goldstine (1977), §2.4. Cauchy (1847) describes the arithmetic Newton iteration for finding, from a root of

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

an integer polynomial modulo m, roots modulo m^2, m^3, ... Bach & Sorenson (1993) and Bernstein (1998b) present efficient tests whether an integer is a perfect power.

9.6. Von Koch (1904) designed a continuous curve which is nowhere differentiable. Three joint copies of it—called a Koch snowflake or a Koch island—form the boundary of the fractal in Figure 9.4. Each of the six white areas around the center "subflake" has again a Koch flake as boundary, and so on forever (or at least up to the resolution).

The total length of the edges grows exponentially fast. At each iteration, the (linear) size of the hexagon shrinks to $1/3$. If we draw the edges of the hexagon, as in Figure 9.3, around the smallest hexagons only, then the total length $6 \cdot l$, say, where l is the length of one edge, is replaced by $7 \cdot 6 \cdot l/3 = 14l$. Starting with length 6 cm, as in Figure 9.4, we get a length of $(14/6)^3 \cdot 6$ cm ≈ 76 cm after three iterations (which is approximated by the little hexagons in the figure). After 83 iterations, the length is more than (current estimates of) the diameter of the universe.

We thank Rob Corless for pointing out the relation of our picture of $\mathbb{Z}_{(7)}$ to von Koch's snowflake.

9.7. The development of BIPOLAR has been discontinued and the system is not available anymore. Von zur Gathen & Gerhard (1996) describe an extension of Cantor's (1989) algorithm. Montgomery (1992) discusses algorithms and implementation results for fast integer arithmetic, in the context of factoring with the elliptic curve method.

Exercises.

9.1 Use Newton iteration to compute $f^{-1} \bmod x^8$ for $f = x^2 - 2x + 1 \in \mathbb{Q}[x]$.

9.2 Compute $94^{-1} \bmod 6561$ using Newton iteration.

9.3 Let $a = x^7 + 2x^4 - 1$ and $b = x^3 + 2x^2 - 3x - 1$ in $\mathbb{Q}[x]$. Compute the quotient and remainder of the division of a by b. Trace by hand the "fast" algorithm for division with remainder on this example.

9.4$^{\longrightarrow}$ Let $a = 30x^7 + 31x^6 + 32x^5 + 33x^4 + 34x^3 + 35x^2 + 36x + 37$ and $b = 17x^3 + 18x^2 + 19x + 20$ in $\mathbb{F}_{101}[x]$, and $f \in \mathbb{F}_{101}[x]$ the reversal of b.

(i) Compute $f^{-1} \bmod x^4$.

(ii) Use (i) to find $q, r \in \mathbb{F}_{101}[x]$ with $a = qb + r$ and $\deg r < 3$.

(iii) Use the Extended Euclidean Algorithm to find $a^{-1} \bmod b$, that is, a polynomial $c \in \mathbb{F}_{101}[x]$ of degree less than 3 with $ac \equiv 1 \bmod b$.

(iv) Use Newton iteration to find $a^{-1} \bmod b^4$.

9.5 Let D be a ring (commutative, with 1) and $f, g \in D[x]$ monic of degree $n > 0$.

(i) Prove that $\operatorname{rev}(fg)^{-1} \operatorname{rem} x^{2n}$ can be computed from $\operatorname{rev}(f)^{-1} \operatorname{rem} x^n$, $\operatorname{rev}(g)^{-1} \operatorname{rem} x^n$, and fg using $2\mathsf{M}(n) + \mathsf{M}(2n) + O(n)$ arithmetic operations in D.

(ii) Prove that $\operatorname{rev}(f)^{-1} \operatorname{rem} x^n$ can be computed from $\operatorname{rev}(fg)^{-1} \operatorname{rem} x^{2n}$ using $\mathsf{M}(n) + O(n)$ operations in D.

9.6* Consider the following variant of the Newton inversion algorithm 9.3. Instead of computing $f^{-1} \bmod x^{2^i}$ for $i = 1, 2, \ldots$, compute the inverse modulo $x^{\lceil l/2^r \rceil}, x^{\lceil l/2^{r-1} \rceil}, \ldots, x^{\lceil l/2 \rceil}, x^l$. Show that the cost of this algorithm is at most $l + \sum_{1 \le j \le r} (\mathsf{M}(\lceil l2^{-j} \rceil) + \mathsf{M}(\lceil l2^{-j-1} \rceil))$. Use $\lceil l2^{-j} \rceil \le \lfloor l2^{-j} \rfloor + 1$ for all j and Exercise 8.34 to conclude that the overall cost is at most $3\mathsf{M}(l) + O(l)$.

9.7 Let D be a ring (commutative, with 1), $R = D[x]$, $p \in R$ monic nonconstant, $r \in \mathbb{N}$, and $f \in R$ of degree less than $n = 2^r \deg p$.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

(i) Show that $p^2, p^4, \ldots, p^{2^r}$ can be computed with $\mathsf{M}(n) + O(n)$ ring operations in D.

(ii) Prove that given the polynomials from (i), $\operatorname{rev}(p)^{-1} \operatorname{rem} x^{\deg p}$, $\operatorname{rev}(p^2)^{-1} \operatorname{rem} x^{2\deg p}$, …, $\operatorname{rev}(p^{2^r})^{-1} \operatorname{rem} x^n$ can be computed using at most $4\mathsf{M}(n) + O(n)$ operations in D. Hint: Exercise 9.5.

(iii) Given the data from (i) and (ii), show that $f \operatorname{rem} p^{2^{r-1}}, f \operatorname{rem} p^{2^{r-2}}, \ldots, f \operatorname{rem} p^2, f \operatorname{rem} p$ can be computed with $2\mathsf{M}(n) + O(n)$ operations in D.

(iv) Show that when $R = \mathbb{Z}$ and $f, p \in \mathbb{N}$ with $f < 2^r p$, you can compute $p^2, p^4, \ldots, p^{2^r}$ and $f \operatorname{rem} p^{2^{r-1}}, f \operatorname{rem} p^{2^{r-2}}, \ldots, f \operatorname{rem} p^2, f \operatorname{rem} p$ using $O(\mathsf{M}(2^r \log p))$ word operations.

9.8 (i) Prove that the Newton inversion algorithm 9.3 works correctly as specified.

(ii) Use Exercise 9.7 to show that the algorithm takes $14\mathsf{M}(l \deg p) + O(l \deg p)$ ring operations in D if $R = D[x]$ for a (commutative) ring D, p is monic, l is a power of 2, and $\deg f < l \deg p$, and $O(\mathsf{M}(l \log p))$ word operations if $R = \mathbb{Z}$ and $|f| < p^l$.

9.9 We consider the linear variant of the Newton inversion algorithm 9.3, where the inverse is computed successively modulo $x^2, x^3, x^4, \ldots, x^l$. If g_i is the inverse modulo x^i, give an explicit formula for the coefficient of x^i in g_{i+1} in terms of the coefficients of g_i and the first $i+1$ coefficients of f. Show that this algorithm takes $O(l^2)$ ring operations.

9.10 Show that the cost of the Newton inversion algorithm 9.3 drops to at most $2\mathsf{M}(l) + 2l$ arithmetic operations if $\operatorname{char} D = 2$.

9.11* Let D be a (commutative) ring, $k \in \mathbb{N}_{>0}$, and $f, g \in D[x]$ with $f(0) = 1$ and $fg \equiv 1 \bmod x^k$.

(i) Let $d \in \mathbb{N}$, $e = 1 - fg$, and $h = g \cdot (e^{d-1} + e^{d-2} + \cdots + e + 1)$. Prove that $fh \equiv 1 \bmod x^{dk}$.

(ii) Letting $d = 2$ gives precisely Algorithm 9.3. State an algorithm for Newton inversion modulo x^l with cubic convergence (that is, $d = 3$), and analyze its cost when l is a power of 3.

9.12* This exercise discusses an alternative to the fast division algorithm 9.5 for computing in residue class rings. It is an adaption of Montgomery's (1985) integer algorithm to polynomials. We let F be a field and $f, r \in F[x]$ such that f is nonconstant, $\deg r < \deg f = n$, and f and r are coprime. For $a \in F[x]$, we represent the residue class $a \bmod f \in R = F[x]/\langle f \rangle$ by the polynomial $a^* = ra \operatorname{rem} f \in F[x]$. This is particularly useful when performing a long computation in R, for example, a modular exponentiation.

(i) Show that $(a+b)^* = a^* + b^*$ and $(ab)^* \equiv r^{-1} a^* b^* \bmod f$ for all $a, b \in F[x]$.

(ii) Let $s \in F[x]$ of degree less than n be the inverse of f modulo r, so that $sf \equiv 1 \bmod r$. Consider the following algorithm for computing $(ab)^*$ from a^* and b^*.

ALGORITHM 9.35 Montgomery multiplication.
Input: $a^*, b^* \in F[x]$ of degrees less than n.
Output: $(ab)^* \in F[x]$.

1. $u \longleftarrow a^* b^*, \quad v \longleftarrow u \operatorname{rem} r$
2. $w \longleftarrow vs \operatorname{rem} r, \quad c^* \longleftarrow (u - wf)/r$
3. **return** c^*

Prove that r divides $u - wf$ in step 2. Conclude that the algorithm works correctly, so that $\deg c^* < n$ and $c^* \equiv r^{-1} a^* b^* \bmod f$, if $\deg r = n - 1$.

(iii) Now let $r = x^{n-1}$ and show that the algorithm can be executed with $3\mathsf{M}(n) + n$ operations in F. You may ignore the cost for computing s. Compare this to using Newton iteration with precomputation.

(iv) Let $a \in F[x]$ of degree less than n and r as in (iii). Employ the above algorithm to show that a can be computed from a^* using $2\mathsf{M}(n) + n$ operations in F, and that conversely a^* can be computed from a using $3\mathsf{M}(n) + n$ operations if r^* is precomputed.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

9.13 Let F be a field of characteristic different from 2, and $\mathsf{M}(n)$, $\mathsf{I}(n)$, $\mathsf{D}(n)$, $\mathsf{S}(n)$ be the computing times for multiplying two polynomials of degree less than n, computing the inverse of a polynomial modulo x^n, division of a polynomial of degree less than $2n$ by a polynomial of degree n, and squaring a polynomial of degree less than n, respectively. Theorems 9.4 and 9.6 show that $\mathsf{I} \in O(\mathsf{M})$ and $\mathsf{D} \in O(\mathsf{M})$. The purpose of this exercise is to show that all four functions are of the same order of magnitude.

(i) Prove the identity $y^2 = \left(y^{-1} - (y+1)^{-1}\right)^{-1} - y$, and conclude that $\mathsf{S} \in O(\mathsf{I})$.

(ii) Show that $\mathsf{M} \in O(\mathsf{S})$, using the identity $fg = ((f+g)^2 - f^2 - g^2)/2$.

(iii) For a polynomial $b \in R[x]$ of degree n, relate $\mathrm{rev}_n(b)^{-1} \bmod x^n$ to the quotient of x^{2n-1} on division by b, and conclude that $\mathsf{I} \in O(\mathsf{D})$. Conclude that $O(\mathsf{M}) = O(\mathsf{I}) = O(\mathsf{D}) = O(\mathsf{S})$.

9.14* Let $a, b, q \in \mathbb{Z}[x]$ such that $a = qb$, $\deg a = n$, and $\|a\|_\infty \leq A$. Use Mignotte's bound 6.33 and a big prime modular approach to show that q can be computed from a and b using $O^\sim(n(n + \log A))$ word operations. You may ignore the cost for finding a big prime. Use Corollary 11.13 for modular arithmetic. See also Exercises 6.26 and 10.21; the latter discusses the small primes variant.

9.15* Let $a, b \in \mathbb{Z}[x]$ such that $n = \deg a = m + \deg b$, with $n, m \in \mathbb{N}$, b is monic, and $\|a\|_\infty, \|b\|_\infty < 2^l$.

(i) Let $f = \mathrm{rev}_{\deg b}(b) \in \mathbb{Z}[x]$. Prove that $\|g_i\|_\infty < 2^{2(i-1)+l}\|g_{i-1}\|_\infty^2$ for $1 \leq i \leq r$ in the Newton inversion algorithm 9.3.

(ii) Prove that $\sum_{0 \leq j < i} j 2^{-j} \leq 2$ for all $i \in \mathbb{N}$. Hint: Consider the formal derivative of the polynomial $\sum_{0 \leq j < i} x^j = (1 - x^i)/(1 - x) \in \mathbb{Z}[x]$.

(iii) Let $S(i) = \log \|g_i\|_\infty$ for $0 \leq i \leq r$. Conclude from (i) and (ii) that $S(i) \leq (2 + l)2^i \in O(nl)$ for all i.

(iv) Perform a similar analysis when $a, b \in R[y][x]$ are bivariate polynomials over a (commutative) ring R and b is monic with respect to x.

9.16 This exercise discusses division with remainder when the degrees of the divisor and the quotient differ significantly. Let $k, m \in \mathbb{N}$ be positive. We consider univariate polynomials over an arbitrary ring (commutative, with 1, as usual).

(i) Prove that division with remainder of a polynomial a of degree less than km by a monic polynomial b of degree m can be done in time $(2k+1)\mathsf{M}(m) + O(km)$. Hint: Partition the dividend a into blocks of size m, and compute $\mathrm{rev}_m(b)^{-1} \bmod x^m$ only once.

(ii) Prove that dividing a polynomial of degree $n < km$ by a monic polynomial of degree $n - m$ takes at most $(k+3)\mathsf{M}(m) + O(km)$ ring operations. Hint: Exercise 8.35.

Determine a small value for the constant in the "O" in both cases.

9.17 Trace the generalized Taylor expansion algorithm 9.14 on computing the (x^2+1)-adic expansion of x^{15} in $\mathbb{Q}[x]$.

9.18 Use the integer variant of Algorithm 9.14 to convert the decimal integer 64 180 into hexadecimal.

9.19 This exercise discusses a divide-and-conquer variant of Horner's rule for computing Taylor expansions. Let R be a ring (commutative, with 1), $u \in R$, $n = 2^k \in \mathbb{N}$ a power of 2, and $a \in R[x]$ of degree less than n. By writing $a = a_1 x^{n/2} + a_0$ with $a_0, a_1 \in R[x]$ of degree less than $n/2$, devise a recursive algorithm which computes $a(x+u)$ and $(x+u)^n$ and takes at most $(c\mathsf{M}(n) + O(n))\log n$ ring operations for some constant c. (The coefficients of $a(x+u)$ are the coefficients in the Taylor expansion of a around u, by Section 5.6.) Determine a small value for c, and compare your result to Corollary 9.16.

9.20 Let R be a ring (commutative, with 1) and $a, p \in R[x]$ with $\deg p = m$ and $\deg a < km$ for some $k, m \in \mathbb{N}$. Prove that the coefficients of a can be computed from its p-adic expansion (4) using at most $(\frac{1}{2}\mathsf{M}(km) + O(km))(1 + \log k)$ ring operations when k is a power of 2.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

9.21 Prove Theorem 9.17.

9.22 Let R be a ring (commutative, with 1), $f \in R[x]$, $u \in R$ a root of f, and $g = f/(x-u)$. Prove that $f'(u) = g(u)$.

9.23 Let R be a ring (commutative, with 1), $f \in R[x]$, and $m \in R$. Prove that $(f \bmod m)' = f' \bmod m$.

9.24 Let R be a ring (commutative, with 1), $f, g \in R[x]$, and $n \in \mathbb{N}$. Prove that $f \equiv g \bmod x^{n+1}$ implies $f' \equiv g' \bmod x^n$, and give an example where $f' \equiv g' \bmod x^{n+1}$ does not hold.

9.25 Let $n \in \mathbb{N}$, R be a ring such that $n!$ is a unit in R, $f \in R[x]$ of degree n, and $f = \sum_{0 \le i \le n} f_i \cdot (x-u)^i$ its Taylor expansion around some $u \in R$. Prove that $f_i = f^{(i)}(u)/i!$ for all i, where $f^{(i)}$ is the ith derivative of f.

9.26 Let R be a ring (commutative, with 1, as usual). For $k \in \mathbb{N}$, the kth **Hasse–Teichmüller derivative** $f^{[k]}$ of a polynomial $\sum_{0 \le i \le n} f_i x^i \in R[x]$ is defined as

$$f^{[k]} = \sum_{k \le i \le n} f_i \binom{i}{k} x^{i-k} \in R[x].$$

Let y be another indeterminate. Show that f has the Taylor expansion $f(x) = \sum_{0 \le i \le n} f^{[i]}(y) \cdot (x-y)^i$ around y.

9.27 Let R be a ring (commutative, with 1), $f_1, \ldots, f_r \in R[x]$ and $e_1, \ldots, e_r, n \in \mathbb{N}_{\ge 1}$. You are to prove three generalizations of the Leibniz rule.

(i) $(f_1 f_2)^{(n)} = \sum_{0 \le i \le n} \binom{n}{i} f_1^{(i)} f_2^{(n-i)}$, where $^{(i)}$ denotes the ith derivative,

(ii) $(f_1 \cdots f_r)' = \sum_{1 \le i \le r} f_i' \prod_{j \ne i} f_j$,

(iii) $(f_1^{e_1} \cdots f_r^{e_r})' = \sum_{1 \le i \le r} e_i f_i' f_i^{e_i - 1} \prod_{j \ne i} f_j^{e_j}$.

(iv) Conclude from (ii) that

$$\frac{f'}{f} = \frac{f_1'}{f_1} + \cdots + \frac{f_r'}{f_r}$$

is the partial fraction decomposition of f'/f, for $f = f_1 \cdots f_r$.

9.28 Compute the first 16 decimal digits of the real root of $y^3 - 2y - 5$ using Newton iteration and $y_0 = 2$ as your starting value. Compare your results with Newton's (page 219). What are the other two roots?

9.29 Under which condition does the Newton iteration algorithm 9.22 work for a rational function $\varphi \in R(y)$? The Newton formula for $\varphi = 1/y - f \in R(y)$ gives exactly the inversion procedure from Theorem 9.2. Why does the polynomial $\varphi = fy - 1 \in R[y]$ *not* work directly?

9.30 Let $\varphi = x^4 + 25x^3 + 129x^2 + 60x + 108 \in \mathbb{Z}[x]$ and $p = 5$.

(i) Determine all roots of φ mod p in $\mathbb{F}_p$.

(ii) Find an a priori bound B such that every root $a \in \mathbb{Z}$ of φ has $|a| \le B$.

(iii) Choose $l \in \mathbb{N}$ such that $2B < p^l$, and apply p-adic Newton iteration to all modular roots of φ from (i).

(iv) Use the results from (iii) to find all roots of φ in $\mathbb{Z}$.

9.31* Let $R = D[x]$ for a (commutative) ring D, and φ, p, l, g_0 be inputs to the p-adic Newton iteration 9.22 with p monic nonconstant, $\deg g_0 < \deg p$, $\deg_x \varphi < l \deg p$, and $\deg_y \varphi = n$. Show that Algorithm 9.22 takes $O(n\mathsf{M}(l \deg p))$ operations in D. Hint: Exercise 9.7.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

9.32* Prove Theorem 9.26. Hint: Exercise 9.7.

9.33* In this exercise, we consider the **linearly convergent Newton iteration**. Let R be a ring (commutative, with 1), $p \in R$, $\varphi \in R[y]$, and $s, g \in \mathbb{R}$ such that $\varphi(g) \equiv 0 \bmod p^k$ for some $k \in \mathbb{N}$ and $s\varphi'(g) \equiv 1 \bmod p$. We define $h \in R$ by the Newton formula

$$h \equiv g - s \cdot \varphi(g) \bmod p^{k+1}.$$

Prove that $\varphi(h) \equiv 0 \bmod p^{k+1}$, $h \equiv g \bmod p^k$, and $s\varphi'(h) \equiv 1 \bmod p$. Derive a linearly convergent analog of Algorithm 9.22 from this, and show that when $R = D[x]$ for a ring D and $p = x$, it takes $O(nl^2)$ operations in D. This is slower than the quadratically convergent variant, but has the advantage that the inverse of the derivative need not be updated.

9.34 Derive the formula

$$g_{i+1} = \frac{1}{2}\left(g_i + \frac{a}{g_i}\right)$$

for $i \geq 0$, which was already known to the Babylonians, and is the Newton iteration for approximating the square root of a. Using this formula, compute a square root of 2 modulo 3^8. What is the corresponding formula for computing an nth root of a?

9.35 Find the Newton formula for approximating $1/\sqrt{a}$. What is the remarkable difference to the Newton formula for $\sqrt{a}$?

9.36 Compute a square root $g \in \mathbb{Q}[x]$ of $f = 1 + 4x \in \mathbb{Q}[x]$ modulo x^8 such that $g(0) = 1$, using Newton iteration.

9.37 Compute a cube root of 2 modulo 625, that is, $g \in \{0, \ldots, 624\}$ such that $g^3 \equiv 2 \bmod 625$. How many such g are there?

9.38 Consider the three prime numbers $p = 5, 7$, and 17. We want to calculate p-adic approximations to $\sqrt{2}$.

(i) For which of the three p does 2 have a square root modulo arbitrary powers of p?

(ii) For those p where possible, compute all square roots of 2 modulo p^6.

9.39 Let $a \in \mathbb{N}_{>0}$ be of word length l, such that $a < 2^{64l}$. For $n \in \mathbb{N}$, we denote by $T(n)$ the number of word operations to compute a^n using repeated squaring. Prove that $T(n) \leq T(\lfloor n/2 \rfloor) + O(\mathsf{M}(nl))$ if $n > 1$, and conclude that $T(n) \in O(\mathsf{M}(nl))$. What is the corresponding result when a is a univariate polynomial over a (commutative) ring R?

9.40$^{\longrightarrow}$ For $n \in \mathbb{N}_{\geq 2}$ and $a \in \mathbb{Z}$ let $S_n(a)$ be the number of solutions $g \in \{0, \ldots, n-1\}$ of the quadratic congruence $g^2 \equiv a \bmod n$.

(i) Which values for $S_p(a)$ are possible when p is prime? Distinguish the three cases $p = 2$, $p \mid a$ and $2 \neq p \nmid a$.

(ii) Let $p \neq 2$ be prime and $e \in \mathbb{N}_{>0}$. Show that $S_{p^e}(a) = S_p(a)$ if $p \nmid a$, and give a counterexample when $p \mid a$.

(iii) Now let n be an odd integer and $n = p_1^{e_1} \cdot \ldots \cdot p_r^{e_r}$ its prime factorization, with distinct primes $p_1, \ldots, p_r \in \mathbb{N}$ and positive integers $e_1, \ldots, e_r$. Find a formula expressing $S_n(a)$ in terms of $S_{p_1}(a), \ldots, S_{p_r}(a)$ in the case where a and n are coprime. Hint: Chinese Remainder Theorem. Conclude that $S_n(1) = 2^r$.

(iv) Which of the numbers 10001, 42814, 31027, 17329 have square roots modulo 50625?

(v) Compute all square roots of 91 modulo 2025 and of 1 modulo 50625.

9.41* For $n \in \mathbb{N}_{\geq 2}$ and $a \in \mathbb{Z}$ let $C_n(a)$ be the number of solutions $g \in \{0, \ldots, n-1\}$ of the cubic congruence $g^3 \equiv a \bmod n$.

(i) Show that the following hold for an odd prime p:

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

- $C_p(a) \le 3$,
- $C_p(a) = 1$ if $p \mid a$ or $p = 3$,
- $C_p(a) \ne 2$, and for any value $C \in \{0,1,3\}$ there is an odd prime p and an integer a such that $3 \ne p \nmid a$ and $C_p(a) = C$.

(ii) Let $p > 3$ be a prime and $e \in \mathbb{N}_{>0}$. Show that $C_{p^e}(a) = C_p(a)$ if $p \nmid a$, and give a counterexample when $p \mid a$.

(iii) Now let $n \in \mathbb{N}$ such that $\gcd(n,6) = 1$, and let $n = p_1^{e_1} \cdot \ldots \cdot p_r^{e_r}$ be its prime factorization, with distinct primes $p_1, \ldots, p_r \in \mathbb{N}$ and positive integers $e_1, \ldots, e_r$. Find a formula expressing $C_n(a)$ in terms of $C_{p_1}(a), \ldots, C_{p_r}(a)$ in the case where a and n are coprime.

(iv) Compute all cube roots of 11 modulo 225 625.

9.42 Let $n \in \mathbb{N}_{>0}$. How many cube roots $g \in \mathbb{F}_7[x]$ modulo x^n of degree less than n does $f = -x^3 + x^2 - x + 1 \in \mathbb{F}_7[x]$ have, and how can they be computed? Compute one for $n = 4$.

9.43* Modify the algorithm for computing nth roots in $\mathbb{Z}$ so as to work when n is a power of 2, by using a 3-adic Newton iteration. Prove that your algorithm is correct, and show that is uses $O(\mathsf{M}(l))$ word operations on inputs of length l. Apply your algorithm to compute $\sqrt[4]{2\,313\,441}$.

9.44* Design a test whether $a \in \mathbb{N}$ is a perfect power. Your test should output $b, d, e, r \in \mathbb{N}$ such that $a = 2^d 3^e b^r$, $\gcd(b,6) = 1$, and r is maximal, using $O(\log a \cdot \mathsf{M}(\log a))$ word operations.

9.45 Let R be a ring (commutative, with 1) with a valuation v, with the special property that $v(a) \le 1$ for all $a \in R$. Show that if $a \in R$ is a unit, then $v(a) = 1$.

9.46 Let R be an integral domain with a valuation v, and K the field of fractions of R. Show that $w(a/b) = v(a)/v(b)$ defines a valuation w on K.

9.47 Conclude the proof of Lemma 9.34.

9.48* Let F be a field, and $v: F[[x]] \longrightarrow \mathbb{R}$ be the x-adic valuation on the ring $F[[x]]$ of formal power series.

(i) For $n \in \mathbb{N}$, let $f_n = 1 + x + \cdots + x^{2n} - x^{2n+1} \in F[[x]]$. Show that $f_0, f_1, \cdots$ is a Cauchy sequence, so that

$$\forall \varepsilon > 0 \quad \exists N \in \mathbb{N} \quad \forall n, m > N \quad v(f_n - f_m) \le \varepsilon.$$

(ii) Prove that the sequence has a limit in $F[[x]]$, so that there exists $f \in F[[x]]$ with

$$\forall \varepsilon > 0 \quad \exists N \in \mathbb{N} \quad \forall n > N \quad v(f - f_n) \le \varepsilon.$$

(iii) Prove that every Cauchy sequence in $F[[x]]$ has a limit in $F[[x]]$, so that $F[[x]]$ is **complete**. Show that $F[x]$ with the x-adic valuation does not have this property. (In fact, $F[[x]]$ can be obtained from $F[x]$ by the same process of "completion" by which one obtains $\mathbb{R}$ from $\mathbb{Q}$ with respect to the absolute value.)

(iv) Let $f = a_0 + a_1 x + \cdots \in F[[x]]$, and $a_0 = 0$. Prove that f does not have an inverse in $F[[x]]$.

(v) Let $f = a_0 + a_1 x + \cdots \in F[[x]]$, and $a_0 \ne 0$. Use Newton iteration to prove that f has an inverse in $F[[x]]$.

9.49 (Aho, Steiglitz & Ullman 1975; see also Schönhage, Grotefeld & Vetter 1994, page 284) In this exercise, you are to improve the cost estimate of Corollary 9.16 by a factor of $\log n$. Let $n \in \mathbb{N}$, R a ring such that $(n-1)!$ is a unit in R, $u \in R$, and $a = \sum_{0 \le i < n} \in R[x]$. Moreover, let $f = \sum_{0 \le i < n} i!\, a_i x^{n-1-i}$ and $g = \sum_{0 \le j < n} u^j x^j / j!$. Show that the coefficient of x^k in the polynomial $a(x+u)$ is equal to $1/k!$ times the coefficient of x^{n-1-k} in the product polynomial fg, for $0 \le k < n$. Conclude that the coefficients of $a(x+u)$, or equivalently, the coefficients of the Taylor expansion of a around u, by Section 5.6, can be computed using $\mathsf{M}(n) + 5n$ arithmetic operations in R.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press

The second concept is the asymptotic behavior of the number of operations. This was not significant for small N so the importance of early forms of the FFT algorithms was not noticed even where they would have been very useful.

James William Cooley (1987)

Il y a une imagination étonnante dans les mathématiques. [...] Il y avait beaucoup plus d'imagination dans la tête d'Archimède que dans celle d'Homère.[1]

Voltaire (1771)

Leibnitz [sic!] crut voir l'image de la création, dans son arithmétique binaire où il n'employait que les deux caractères zéro et l'unité. Il imagina que l'unité pouvait représenter Dieu, et zéro le néant; et que l'Être suprême avait tiré du néant tous les êtres, comme l'unité avec le zéro exprime tous les nombres dans ce système d'arithmétique.[2]

Pierre Simon Laplace (1812)

Guided by an instinctive sense of the beautiful and fitting, in a happy moment I have succeeded in grasping this much wished for representation, with which I propose now and for ever to take my farewell of this long and deeply excogitated theorem.

James Joseph Sylvester (1853)

[1] There is an astonishing imagination in mathematics. [...] There was far more imagination in the head of Archimedes than in that of Homer.

[2] Leibniz believed he saw the image of creation in his binary arithmetic in which he employed only the two characters, zero and unity. He imagined that unity can represent God, and zero nothing; and that the Supreme Being might have drawn all beings from nothing, just as unity with zero expresses in this binary arithmetic all numbers.

https://doi.org/10.1017/CBO9781139856065.012 Published online by Cambridge University Press